



UNIVERSITEIT VAN AMSTERDAM

Stochastic Simulation

NOTES BY

Hengyi Li

LECTURERS

Dr. J. L. Dorsman
Prof. dr. J. M. Mooij

INTRODUCTION. The field of advanced simulation contains powerful tools and techniques to study stochastic processes and other objects that defy direct mathematical analysis. This course gives a broad treatment of the important aspects of stochastic simulation and its applications e.g. queueing, reliability, manufacturing, risk analysis, and financial models.

Contents

1	Introduction	1
2	Generating Random Objects	3
2.1	Uniform Numbers	3
2.2	Generally Distributed Numbers	4
2.3	Random Vectors	6
2.4	Elementary Stochastic Processes	7
3	Discrete-Event Simulation	11
4	Output Analysis	14
5	Steady-State Simulation	21
5.1	Regenerative Method	21
5.2	Batch-Means Method	23
6	Variance Reduction Methods	26
6.1	Control Variates	26
6.2	Antithetic Sampling	28
6.3	Conditional Monte Carlo	29
6.4	Splitting	30
6.5	Common Random Numbers	32
6.6	Stratification	33
6.7	Indirect Sampling	35
6.8	Importance Sampling	36
7	Derivative Estimation	49
7.1	Finite Differences	49
7.2	Infinitesimal Perturbation Analysis	51
7.3	Likelihood Ratio Method	52

8	Simulation-Based Optimisation	60
9	Rare-Event Simulation	65
	References	68

Introduction

Complex stochastic dynamical systems of the *discrete event type*. Focus on systems in which no explicit results for performance measures are available. As an alternative, these systems are *simulated*. After repeating this experiment (to obtain the output of multiple *runs*), statistical claims can be made about the value of the performance measure.

EXAMPLE 1.1 (Networks of Queuing Systems).

EXAMPLE 1.2 (Multivariate Ruin Models).

Exercises

EXERCISE 1.1. Draw n independent samples of $U = (U_1, U_2)$, where U_1 is uniform on $[-1, 1]$, U_2 is uniform on $[0, 1]$, and the coordinates are independent. For sufficiently large n , compute

$$\frac{4}{n} \sum_{i=1}^n \mathbb{1} \left\{ (U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1 \right\}.$$

What value do you obtain? Explain your findings.

SOLUTION.

```
import numpy as np
np.random.seed(12345)

n = 1000

U1 = np.random.uniform(-1, 1, n)
U2 = np.random.uniform(0, 1, n)
U = list(zip(U1, U2))

print(U) # end of part 1

inside_circle = np.sum((U1**2 + U2**2) <= 1)
result = 4 * inside_circle / n

print(result) # end of part 2
```

The estimator approximates π . The idea is akin to randomly placing points within a coordinate system with $x = [-1, 1]$ and $y = [0, 1]$. If the distance from a point to the origin is less than or equal to 1, it's counted as 1. Consequently, when multiplied by the appropriate coefficient 4, we obtain an approximation of π . $\square$

Generating Random Objects

| Uniform Numbers

SECTION 2.2

| Generally Distributed Numbers

Binomial and Geometric

EXAMPLE 2.1 (Binomial Distribution). Let X be binomial with parameters n and p . Let $X_i := 1$ if $U_i < p$ and 0 else. Then $\sum_{i=1}^n X_i$ has the desired distribution.

The negative binomial distribution can be sampled from similarly.

EXAMPLE 2.2 (Geometric Distribution). Let X be geometric with parameter p . Sample U_i until $U_i < p$; let us say this happens at the N -th attempt. Then $N - 1$ has the desired distribution if you only count the number of failures (or N if you also count the success).

Exponential, Poisson, Weibull, and Pareto

This algorithm is a simple form of *inversion*. Relies on the left-continuous version of the inverse distribution function: with $F(\cdot)$ the distribution function of the random variable X ,

$$F^{\leftarrow}(u) := \min \{x : F(x) \geq u\},$$

where $F^{\leftarrow}(u)$ is known as the quantile function.

PROPOSITION 2.3.

EXAMPLE 2.4. Generate exponentially distributed numbers (with mean λ^{-1}) from U :

$$F(x) = 1 - e^{-\lambda x} \iff F^{\leftarrow}(u) = -\frac{\log(1-u)}{\lambda}.$$

Then $-\frac{\log(1-U)}{\lambda}$ has an $\exp(\lambda)$ distribution, and we may as well use $-\frac{\log U}{\lambda}$. Erlang(k, λ) now follows directly as well.

Normal and Lognormal

Acceptance-rejection method is to be used when no explicit expression for CDF is available, or the CDF does not allow explicit inversion.

Setup. X has PDF $f(x)$, and Y has PDF $g(x)$ such that, for some finite constant $C > 1$ and all x , $f(x) \leq Cg(x)$. We know how to sample Y , but we don't know how to sample X .

Algorithm. Sample Y , and in addition U that is uniform on $[0, 1]$. If

$$U \leq \frac{f(Y)}{Cg(Y)},$$

then $X := Y$ (accept), and otherwise we reject the sample and try again.

Now we have to show that an accepted sample indeed has the same CDF $F(x)$ as X .

Proof. Define the event of acceptance by

$$A := \left\{ U \leq \frac{f(Y)}{Cg(Y)} \right\}.$$

The accepted sample has distribution $\mathbb{P}(Y \leq x \mid A)$. Since U and Y are independent,

$$\mathbb{P}(A) = \int \frac{f(y)}{Cg(y)} g(y) dy = \frac{1}{C},$$

and

$$\mathbb{P}(Y \leq x, A) = \frac{1}{C} \int_{-\infty}^x f(y) dy = \frac{F(x)}{C}.$$

Dividing gives $\mathbb{P}(Y \leq x \mid A) = F(x)$. □

THEOREM 2.5 (Box–Muller Transform).

With U_1 and U_2 independent uniforms on $[0, 1]$,

$$Y_1 := \sqrt{-2 \log U_1} \sin(2\pi U_2), \quad Y_2 := \sqrt{-2 \log U_1} \cos(2\pi U_2)$$

yields two independent standard normal random variables.

Proof. First observe

$$U_1 = U_1(Y_1, Y_2) = e^{-\frac{Y_1^2 + Y_2^2}{2}}, \quad U_2 = U_2(Y_1, Y_2) = \frac{1}{2\pi} \text{atan2}(Y_1, Y_2) \pmod{1}.$$

Hence, with $J(y_1, y_2)$ the determinant of the Jacobian,

$$f_{Y_1, Y_2}(y_1, y_2) = |J(y_1, y_2)| f_{U_1, U_2}(u_1(y_1, y_2), u_2(y_1, y_2)) = |J(y_1, y_2)|$$

since $f_{U_1, U_2}(u_1(y_1, y_2), u_2(y_1, y_2)) = 1$. Then we calculate

$$\begin{aligned} \frac{du_1}{dy_1} &= -y_1 u_1, & \frac{du_2}{dy_1} &= \frac{1}{2\pi} \frac{y_2}{y_1^2 + y_2^2}, \\ \frac{du_1}{dy_2} &= -y_2 u_1, & \frac{du_2}{dy_2} &= \frac{1}{2\pi} \frac{-y_1}{y_1^2 + y_2^2}. \end{aligned}$$

The absolute determinant is $\frac{1}{2\pi} e^{-\frac{y_1^2 + y_2^2}{2}}$. Hence, as desired,

$$f_{Y_1, Y_2}(y_1, y_2) = \frac{1}{2\pi} e^{-\frac{y_1^2 + y_2^2}{2}}.$$

□

SECTION 2.3

| Random Vectors

Multivariate Normal distribution characterised through mean vector $\boldsymbol{\mu}$ (of length p) and covariance matrix Σ (of dimension $p \times p$). Without loss of generality: $\boldsymbol{\mu} = \mathbf{0}$. Σ is positive definite (i.e. $\mathbf{x}^T \Sigma \mathbf{x} > 0$ for any non-zero $\mathbf{x}$), hence can be written as CC^T , for a lower triangular matrix C . Efficient algorithms for such *Cholesky decomposition* exist.

Now the X_i (for $i = 1, \dots, p$) can be sampled as follows. Let $\mathbf{Y}$ be a p -dimensional normal vector with independent standard Normal components (i.e., with covariance matrix I). Let $\mathbf{X} := C\mathbf{Y}$. Then $\mathbf{X}$ has the right covariance matrix $CC^T = \Sigma$. Indeed, since C is lower-triangular, it holds for $i \leq j$ that:

$$\begin{aligned} \text{Cov}(X_i, X_j) &= \text{Cov}\left(\sum_{k=1}^i C_{ik}Y_k, \sum_{l=1}^j C_{jl}Y_l\right) = \sum_{k=1}^i \sum_{l=1}^j C_{ik}C_{jl}\text{Cov}(Y_k, Y_l) \stackrel{l=k}{=} \sum_{k=1}^i C_{ik}C_{jk} \\ &= \sum_{k=1}^i C_{ik}C_{kj}^T = \Sigma_{ij} \end{aligned}$$

as desired.

SECTION 2.4

| Elementary Stochastic Processes

Exercises

EXERCISE 2.1. Write an inversion routine for a Weibull random variable X with $\mathbb{P}(X > x) = e^{-x^\beta}$ for $x > 0$. Check it by plotting a histogram of simulated values against the theoretical density, for example with $\beta = 1/2$.

SOLUTION. From lecture notes, for a Weibull distribution, we have

$$F(x) = \mathbb{P}(X \leq x) = 1 - e^{-\lambda x^\alpha}.$$

Given $\lambda = 1$ and $\alpha = \beta$ in this case, we rewrite

$$F(x) = \mathbb{P}(X \leq x) = 1 - e^{-x^\beta}$$

in this case. By inversion, let $F(x) = U$, we have

$$x = (-\ln(1 - U))^{\frac{1}{\beta}}.$$

The code is given below.

```
import numpy as np
np.random.seed(12345)
import matplotlib.pyplot as plt

def weibull(beta, size=1):
    U = np.random.uniform(0, 1, size)
    X = (-np.log(1 - U))**(1/beta)
    return X

beta = 1/2
n = 10000

samples = weibull(beta, n)

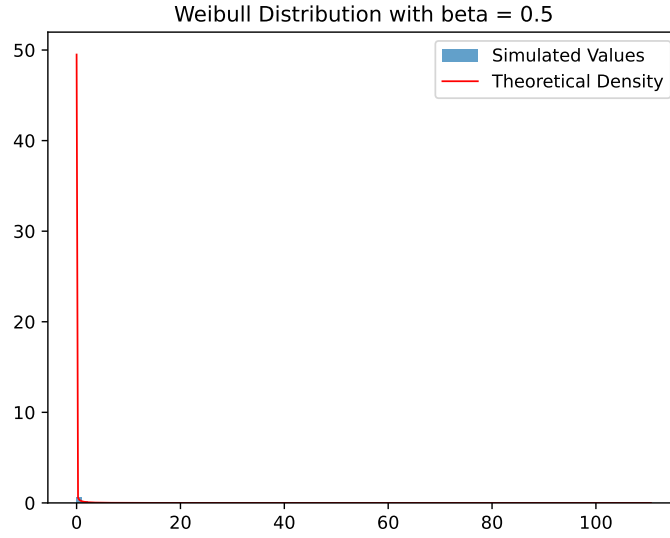
plt.hist(samples, bins=100, density=True, alpha=0.7, label="Simulated Values")
x = np.linspace(1e-4, max(samples), 400)
y = (beta * x**(beta-1) * np.exp(-x**beta))
plt.plot(x, y, 'r-', lw=1, label="Theoretical Density")
plt.legend()
plt.title(f"Weibull Distribution with beta = {beta}")
plt.savefig('HS1_Q2.pdf')
plt.show()
```

□

EXERCISE 2.2. Suppose $f(x) = cg(x)\overline{H}(x)$ is the density of X , where g is the density of Y and $\overline{H}(x) = \mathbb{P}(Z > x)$. Show that X can be generated by sampling Y and Z independently and rejecting until $Y \leq Z$.

SOLUTION. Sample Y and Z independently and accept $X = Y$ if $Y \leq Z$; otherwise repeat. Writing $A = \{Y \leq Z\}$, the unnormalized density of an accepted Y is $g(y)\overline{H}(y)$. Since $f(y) = cg(y)\overline{H}(y)$ integrates to one,

$$\mathbb{P}(A) = \int g(y)\overline{H}(y) dy = \frac{1}{c}.$$



Thus the conditional density is

$$f_{Y|A}(y) = \frac{g(y)\bar{H}(y)}{\mathbb{P}(A)} = c g(y)\bar{H}(y) = f(y).$$

□

EXERCISE 2.3. Use acceptance–rejection to generate an inverse Gaussian random variable with $\xi = c = 1$ and density

$$f(x) = \frac{1}{\sqrt{2\pi}x^3} \exp\left\{1 - \frac{1}{2}\left(x + \frac{1}{x}\right)\right\}, \quad x > 0.$$

Check the routine using confidence intervals for $\mathbb{E}X$, $\mathbb{E}X^2$, and $\text{Var}X$. Use the known formulas $\mathbb{E}X = c/\xi$ and $\text{Var}X = c/\xi^3$.

SOLUTION. Use the exponential proposal $g(y) = \frac{1}{2}e^{-y/2}$, $y > 0$. The ratio

$$\frac{f(y)}{g(y)} = \frac{2e}{\sqrt{2\pi}} y^{-3/2} e^{-1/(2y)}$$

is maximized at $y = 1/3$, with maximum less than 3, so $C = 3$ is valid. The proposal e^{-y} would give an unbounded ratio and is unsuitable. Sample $Y = -2 \log U$ and accept with probability $f(Y)/(Cg(Y))$. The theoretical values are $\mathbb{E}X = 1$, $\mathbb{E}X^2 = 2$ and $\text{Var}(X) = 1$. The variance interval below uses the asymptotic standard error of the sample centered second moment.

```
import numpy as np
from scipy.stats import norm
np.random.seed(12345)

def inverse_gaussian_ar(num_samples):
    samples = []
    C = 3.0
    while len(samples) < num_samples:
        y = -2 * np.log(np.random.uniform())
```

```

    f = np.exp(1 - 0.5 * (1/y + y)) / np.sqrt(2*np.pi*y**3)
    g = 0.5 * np.exp(-y/2)
    if np.random.uniform() <= f / (C*g):
        samples.append(y)
    return np.asarray(samples)

x = inverse_gaussian_ar(100000)
n = len(x)
z = norm.ppf(0.975)
mean_x, mean_x2 = np.mean(x), np.mean(x*x)
variance = np.var(x, ddof=1)
se_mean = np.std(x, ddof=1) / np.sqrt(n)
se_second = np.std(x*x, ddof=1) / np.sqrt(n)
se_variance = np.std((x-mean_x)**2, ddof=1) / np.sqrt(n)
for name, value, se in [("E[X]", mean_x, se_mean), ("E[X^2]", mean_x2,
↪ se_second), ("Var[X]", variance, se_variance)]:
    print(f"{name}: {value:.6f}, 95% CI: ({value-z*se:.6f},
↪ {value+z*se:.6f})")

```

```

E[X]: 1.002285, 95% CI: (0.996086, 1.008483)
E[X^2]: 2.004732, 95% CI: (1.969666, 2.039799)
Var[X]: 1.000168, 95% CI: (0.975260, 1.025075)

```

□

Discrete-Event Simulation

Exercises

EXERCISE 3.1. Write a discrete-event simulation of a single run of the following harbor model. Ships have i.i.d. Weibull interarrival times with mean 1.25 days. The dock has two berths and two cranes. If both berths are occupied, ships wait in a FIFO queue. The unloading time with one crane has a normal $(1, 0.1)$ distribution truncated to $[0.5, 1.5]$.

If there are no other ships in the harbor when a ship reaches a berth, both cranes unload it, halving its unloading time. If another ship moors meanwhile, its unloading waits until the first ship is finished. If other ships are present when a ship arrives, it is unloaded by one crane. Start with an empty harbor at time zero. Estimate the average time ships wait until they are unloaded and the longest waiting time during 90 days. Discuss the reliability of these estimates.

SOLUTION. The Weibull shape is not specified by the question; this example takes shape 1.5 and chooses scale $1.25/\Gamma(1 + 1/1.5)$ to obtain the required mean. The normal parameter 0.1 is interpreted as a variance. Rejection sampling gives the truncated unloading time before it is halved when two cranes are used. Crane requests are awaited before unloading begins. The code reports waiting until unloading starts and residence time until unloading finishes, using ships observed by day 90 for the respective measures.

```
import math
import numpy as np
import simpy
np.random.seed(12345)

def truncated_normal():
    while True:
        u1, u2 = np.random.uniform(size=2)
        z = np.sqrt(-2*np.log(u1))*np.cos(2*np.pi*u2)
        x = 1 + np.sqrt(0.1)*z
        if 0.5 <= x <= 1.5:
            return x

def run_harbor(days, shape=1.5):
    env = simpy.Environment()
    berths = simpy.Resource(env, capacity=2)
    cranes = simpy.Resource(env, capacity=2)
    waits, residence, queue_changes = [], [], [(0.0, 0)]
    scale = 1.25 / math.gamma(1 + 1/shape)

    def record_queue():
        queue_changes.append((env.now, len(berths.queue)))

    def unload():
        arrival = env.now
        with berths.request() as berth:
            record_queue()
            yield berth
            record_queue()
            use_two = berths.count == 1 and not berths.queue and cranes.count
            ↪ == 0
```

```

        requests = [cranes.request() for _ in range(2 if use_two else 1)]
        yield env.all_of(requests)
        waits.append(env.now - arrival)
        yield env.timeout(truncated_normal() / len(requests))
        residence.append(env.now - arrival)
        for request in requests:
            cranes.release(request)

    def arrivals():
        while True:
            u = np.random.uniform()
            yield env.timeout(scale*(-np.log(u))**(1/shape))
            env.process(unload())

    env.process(arrivals())
    env.run(until=days)
    queue_changes.append((days, queue_changes[-1][1]))
    return np.asarray(waits), np.asarray(residence), queue_changes

waiting, residence, _ = run_harbor(90)
print(f"Mean waiting until unloading starts: {np.mean(waiting):.6f} days")
print(f"Longest observed waiting: {np.max(waiting):.6f} days")
print(f"Mean time until unloading finishes: {np.mean(residence):.6f} days")

```

```

Mean waiting until unloading starts: 0.029270 days
Longest observed waiting: 0.564727 days
Mean time until unloading finishes: 0.660981 days

```

One 90-day run has substantial sampling error and excludes ships whose relevant waiting or residence period is not complete by the end. Independent replications or a longer run are needed to assess precision. □

Output Analysis

We want to estimate the performance measure

$$z := \mathbb{E}Z, \quad \hat{z}_R := \frac{1}{R} \sum_{r=1}^R Z_r.$$

performing independent samples $Z_1, \dots, Z_R$ of Z . Two issues play a role: *bias* (whether the expectation equals the parameter of interest) and *accuracy* (the variance of estimator) for the performance of this estimator. When unbiased, we would like to have a minimal variance.

Standard approach: if $\sigma^2 = \text{Var}Z < \infty$, then the *central limit theorem* provided us with: as $R \rightarrow \infty$,

$$\sqrt{R}(\hat{z}_R - z) \rightarrow \mathcal{N}(0, \sigma^2).$$

This suggests for finite R the approximation, with $V \sim \mathcal{N}(0, 1)$,

$$\hat{z}_R \approx z + \frac{\sigma V}{\sqrt{R}}.$$

Remark. (i) The convergence rate is of order $R^{-\frac{1}{2}}$. This is a rather slow convergence rate: if we want our numbers to be 10 times as precise, we must do 100 times as many runs.

(ii)

(iii)

Based on this convergence in distribution, one could use confidence intervals of the type

$$\left(\hat{z}_R - q_\alpha \frac{\sigma}{\sqrt{R}}, \hat{z}_R + q_\alpha \frac{\sigma}{\sqrt{R}} \right),$$

with q_α reflecting the $1 - \frac{1-\alpha}{2}$ -quantile of $\mathcal{N}(0, 1)$, in the sense that

$$1 - \Phi(q_\alpha) = \mathbb{P}(\mathcal{N}(0, 1) \geq q_\alpha) = \frac{1 - \alpha}{2}.$$

For example, confidence level α equalling 0.95 leads to $q_\alpha \approx 1.96$.

To estimate σ^2 , traditional estimator:

$$\hat{\sigma}^2 := \frac{1}{R-1} \sum_{r=1}^R (Z_r - \hat{z}_R)^2 = \frac{1}{R-1} \left(\sum_{r=1}^R Z_r^2 - R\hat{z}_R^2 \right).$$

This is an unbiased estimator.

Proof. Let $Z_1, \dots, Z_R$ be *i.i.d.* samples from Z . □

The confidence intervals give a handle on how to answer the following issue: how many times should we repeat the simulation experiment to obtain an estimate with a given precision?

EXAMPLE 4.1. Suppose that we want to compute an α -confidence interval for $\frac{\mathbb{E}[Z^2]}{2\mathbb{E}[Z]}$. We use the estimator

$$\hat{z} = \frac{\sum_{r=1}^R Z_r^2}{2 \sum_{r=1}^R Z_r}.$$

EXAMPLE 4.2. Estimation of standard deviation of random variable W .

EXAMPLE 4.3. Let A_n and B_n be independent sequences of *i.i.d.* random variables. We might be interested in the value of

$$Y = \sum_{n=0}^{\infty} B_n \prod_{m=1}^n A_m.$$

If the B_i and the A_i are both *i.i.d.*, then

$$\mathbb{E}Y = \frac{\mathbb{E}B}{1 - \mathbb{E}A}.$$

Remark. An application could be the present value of a perpetuity, a security that pays for an infinite amount of time. B_n is then the cash flow in slot n , the A_m is the discount rate in slot m for the purpose of computing the present value.

We wish to estimate

$$\varrho := \mathbb{E}Y = \frac{\mathbb{E}B}{1 - \mathbb{E}A}.$$

Then $\mathbf{z} = (z_1, z_2)^T$, with $z_i = \mathbb{E}Z(i)$, where

$$Z(1) = B, \quad Z(2) = A.$$

Goal is to estimate $\varrho = f(\mathbf{z})$, with

$$f(\mathbf{z}) = \frac{z_1}{1 - z_2}.$$

Again, our aim is to estimate a smooth function of expectations, in this case $\mathbb{E}B$ and $\mathbb{E}A$. We estimate $f(\mathbf{z})$ by $f(\hat{\mathbf{z}}_R)$ applying the *delta-method*. Using the usual Taylor arguments, with $\nabla f(\mathbf{z})$ the row vector of partial derivatives,

$$f(\hat{\mathbf{z}}_R) - f(\mathbf{z}) = \nabla f(\mathbf{z})(\hat{\mathbf{z}}_R - \mathbf{z}) + o(\|\hat{\mathbf{z}}_R - \mathbf{z}\|) = \frac{1}{R} \sum_{r=1}^R V_r + o(\|\hat{\mathbf{z}}_R - \mathbf{z}\|),$$

where V_r denotes $\nabla f(\mathbf{z})(\mathbf{Z}_r - \mathbf{z})$. By *central limit theorem*, with $\sigma^2 := \text{Var}(V_1)$,

$$\sqrt{R} \frac{\frac{1}{R} \sum_{r=1}^R V_r - \mathbb{E}[V_1]}{\sigma} \rightarrow \mathcal{N}(0, 1),$$

or

$$\sqrt{R} \frac{f(\hat{\mathbf{z}}_R) - f(\mathbf{z})}{\sigma} \rightarrow \mathcal{N}(0, 1),$$

which would lead to a confidence interval for $f(\mathbf{z})$:

$$\left(f(\hat{\mathbf{z}}_R) - q_\alpha \frac{\sigma}{\sqrt{R}}, f(\hat{\mathbf{z}}_R) + q_\alpha \frac{\sigma}{\sqrt{R}} \right).$$

To evaluate σ^2 , note that

$$\sigma^2 = \nabla f(\mathbf{z}) \Sigma \nabla f(\mathbf{z})^T,$$

where $\Sigma_{ij} := \text{Cov}(Z_1(i), Z_1(j))$. For $d = 1$ (where d is dimension of vector $\mathbf{Z}_r$), we just get $\sigma^2 = (f'(z))^2 \text{Var}(Z)$; for $f(z) = z$ this gives us back our earlier procedure.

Remark. There are two differences.

- (i) Estimator $f(\hat{\mathbf{z}}_R)$ is generally biased.
- (ii) σ^2 harder to estimate.

EXAMPLE 4.4 (Computing Roots of Equations Defined by Expectations). Let $f : \mathbb{R}^{d+1} \mapsto \mathbb{R}$ be known, $\mathbf{z} := \mathbb{E}\mathbf{Z} \in \mathbb{R}^d$. Then we want to find the θ such that

$$f(\mathbf{z}, \theta) = 0$$

call the root θ^* .

Exercises

EXERCISE 4.1. For i.i.d. replications $Z_1, \dots, Z_R$ with sample mean $\hat{z}$, prove that

$$s^2 = \frac{1}{R-1} \sum_{r=1}^R (Z_r - \hat{z})^2$$

is unbiased for $\text{Var}Z$. Give the bias if the denominator is replaced by R .

SOLUTION. Given the sample mean

$$\hat{z} = \frac{1}{R} \sum_{r=1}^R Z_r,$$

we have the empirical variance:

$$\begin{aligned} s^2 &= \frac{1}{R-1} \left(\sum_{r=1}^R Z_r^2 - R\hat{z}^2 \right) = \frac{1}{R-1} \left(\sum_{r=1}^R Z_r^2 - 2R\hat{z}^2 + R\hat{z}^2 \right) \\ &= \frac{1}{R-1} \left(\sum_{r=1}^R Z_r^2 - 2\hat{z} \sum_{r=1}^R Z_r + \sum_{r=1}^R \hat{z}^2 \right) = \frac{1}{R-1} \left(\sum_{r=1}^R (Z_r - \hat{z})^2 \right) \end{aligned}$$

To show that s^2 is unbiased, we need to compute $\mathbb{E}[s^2]$ and show that it equals the population variance σ^2 , then

$$\mathbb{E}[s^2] = \mathbb{E} \left[\frac{1}{R-1} \left(\sum_{r=1}^R (Z_r - \hat{z})^2 \right) \right] = \frac{1}{R-1} \mathbb{E} \left[\sum_{r=1}^R (Z_r - \hat{z})^2 \right] = \frac{1}{R-1} \sum_{r=1}^R \mathbb{E}[(Z_r - \hat{z})^2].$$

For the expectation of $(Z_r - \hat{z})^2$, we have

$$\mathbb{E}[(Z_r - \hat{z})^2] = \mathbb{E} \left[\left(Z_r - \frac{1}{R} \sum_{r=1}^R Z_r \right)^2 \right] = \mathbb{E}[Z_r^2] - 2\mathbb{E} \left[Z_r \left(\frac{1}{R} \sum_{r=1}^R Z_r \right) \right] + \mathbb{E} \left[\left(\frac{1}{R} \sum_{r=1}^R Z_r \right)^2 \right].$$

To simplify each term, since

$$\mathbb{E}[Z_r] = \mu, \quad \mathbb{E}[Z_r^2] = \sigma^2 + \mu^2, \quad \text{and} \quad \mathbb{E}[Z_i Z_j]_{i \neq j} = \mu^2,$$

we have

$$\begin{aligned} \mathbb{E} \left[Z_r \left(\frac{1}{R} \sum_{r=1}^R Z_r \right) \right] &= \frac{1}{R} \mathbb{E} \left[Z_r \left(\sum_{m=1}^R Z_m \right) \right] = \frac{1}{R} \sum_{m=1}^R \mathbb{E}[Z_r Z_m] = \frac{1}{R} \left(\mathbb{E}[Z_r^2] + \sum_{\substack{m=1 \\ m \neq r}}^R \mathbb{E}[Z_r Z_m] \right) \\ &= \frac{1}{R} [\sigma^2 + \mu^2 + (R-1)\mu^2] = \frac{1}{R} (\sigma^2 + R\mu^2). \end{aligned}$$

$$\begin{aligned} \mathbb{E} \left[\left(\frac{1}{R} \sum_{r=1}^R Z_r \right)^2 \right] &= \frac{1}{R^2} \mathbb{E} \left[\sum_{i=1}^R \sum_{j=1}^R Z_i Z_j \right] = \frac{1}{R^2} \mathbb{E} \left[\sum_{i=1}^R Z_i^2 + \sum_{i=1}^R \sum_{\substack{j=1 \\ j \neq i}}^R Z_i Z_j \right] \\ &= \frac{1}{R^2} [R(\sigma^2 + \mu^2) + R(R-1)\mu^2] = \frac{1}{R} (\sigma^2 + R\mu^2). \end{aligned}$$

Therefore, we plug in these terms back, we have

$$\mathbb{E}[(Z_r - \hat{z})^2] = \sigma^2 + \mu^2 - \frac{2}{R} (\sigma^2 + R\mu^2) + \frac{1}{R} (\sigma^2 + R\mu^2) = \frac{R-1}{R} \sigma^2,$$

then

$$\mathbb{E}[s^2] = \frac{1}{R-1} \sum_{r=1}^R \mathbb{E}[(Z_r - \hat{z})^2] = \sigma^2.$$

Therefore, s^2 is an unbiased estimator of the population variance σ^2 .

Now, consider the biased estimator of the variance:

$$\tilde{s}^2 = \frac{1}{R} \sum_{r=1}^R (Z_r - \hat{z})^2$$

Using the previous result, we have:

$$\mathbb{E}[\tilde{s}^2] = \frac{R-1}{R} \sigma^2.$$

Then the bias is

$$\frac{R-1}{R} \sigma^2 - \sigma^2 = -\frac{\sigma^2}{R}.$$

□

EXERCISE 4.2. Identify the errors in the run-length calculations of Example III.1.2 of [1]. The example estimates π using $Z = 4\mathbb{1}_{\{U_1^2 + U_2^2 < 1\}}$ with independent $\text{uniform}(0, 1)$ coordinates and states $\text{Var}Z \approx 2.70$, followed by the run length $R \approx 1.96^2 2.70^2 / 0.5^2 \approx 112$ and a hundredfold increase for each further decimal place.

In Buffon's needle experiment with parallel lines one unit apart and $0 < L < 1$, how should the needle length L be chosen for maximal efficiency? For that choice, carry out run-length calculations analogous to the example.

SOLUTION. For the quarter-circle estimator, $\text{Var}(Z) = 4\pi - \pi^2 \approx 2.70$. The run-length formula uses this variance once:

$$R \approx \frac{1.96^2 (4\pi - \pi^2)}{\delta^2}.$$

Thus half-width $\delta = 0.5$ needs about 42 replications, rather than 112. Decreasing the half-width by a factor of ten still multiplies the required run length by one hundred. Confidence-interval precision is an approximate probabilistic statement, rather than a guarantee that specified digits are correct. For Buffon's estimator $\hat{\pi} = 2L/\hat{P}$ with unit line spacing, the delta-method asymptotic variance per replication is

$$v_\pi = \pi^2 \left(\frac{\pi}{2L} - 1 \right).$$

It decreases with L , so the best admissible choice approaches $L = 1$. The analogous run length is $R \approx 1.96^2 v_\pi / \delta^2$; at $L = 1$ and $\delta = 0.5$, this is about 87.

In Buffon's Needle experiment, suppose we drop a needle of length L onto a plane surface on which parallel lines are drawn a width t , the probability that the needle will cross one of the lines is given by

$$P = \frac{2L}{\pi t}.$$

For the maximal efficiency, we have to maximise P , since this can reduce the number of trials needed to get an accurate estimate. Given $0 < L < 1$ and $t = 1$, the length L should be as close to 1 as possible.

Regarding the study on run-length considerations, I employed the following code, changing the value of L , and proceeded to generate the accompanying illustrations.

```
import numpy as np
np.random.seed(12345)
```

```

import matplotlib.pyplot as plt

def buffons_needle(num_trials, L):
    t = 1
    count = 0

    estimates = []

    for _ in range(num_trials):
        x = np.random.uniform(0, t/2)
        theta = np.random.uniform(0, np.pi/2)

        if x <= (L/2) * np.sin(theta):
            count += 1

        if count > 0:
            estimate_pi = (2 * L * (_ + 1)) / (t * count)
            estimates.append(estimate_pi)
        else:
            estimates.append(np.nan)

    return estimates

num_trials = 1000
for j, L in enumerate([0.1, 0.3, 0.6, 0.99], 1):
    estimates = buffons_needle(num_trials, L)
    plt.figure()
    plt.plot(np.arange(1, num_trials+1), estimates)
    plt.axhline(np.pi, color='r', linestyle='--')
    plt.xlabel('Number of trials')
    plt.ylabel('Estimate of pi')
    plt.title(f"Needle length L = {L}")
    plt.savefig(f"HS2_Q2_{j}.pdf")
    plt.close()

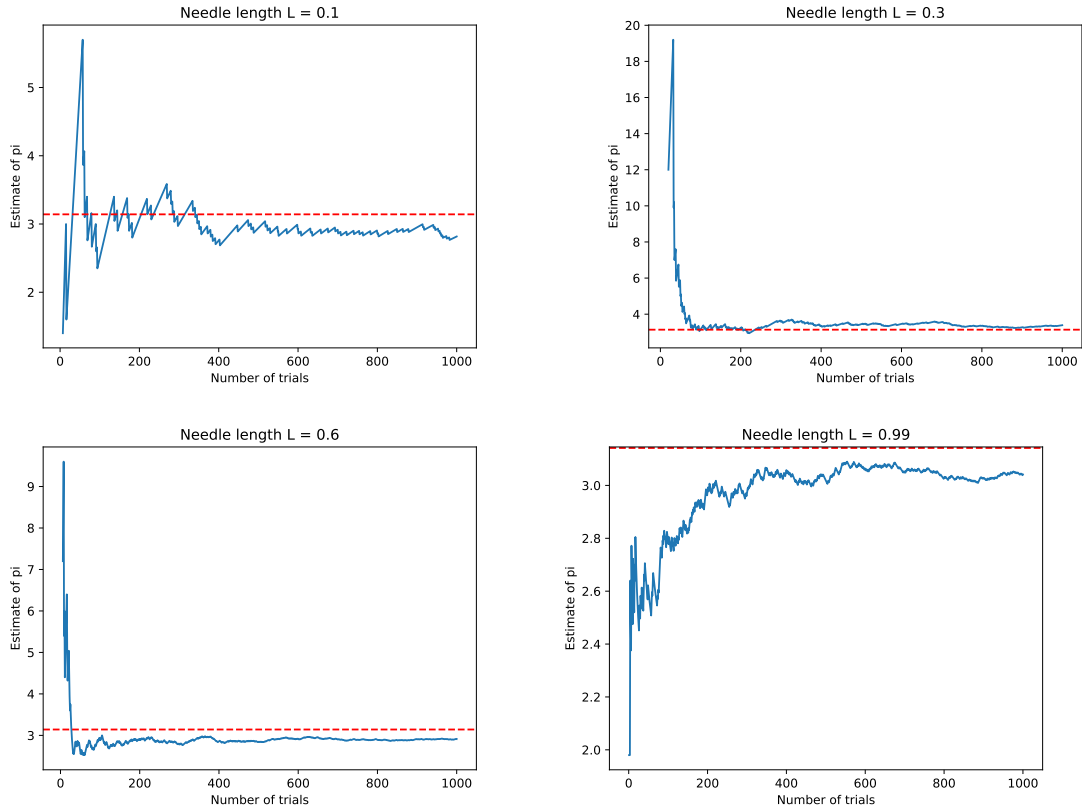
```

It is observable that if the value of L is excessively small, the experiment struggles to accurately simulate the value of π . Furthermore, as the value of L increases, the number of trials required to simulate the value of π diminishes. This corroborates our initial conclusion, suggesting that L should be as close to 1 as possible. $\square$

EXERCISE 4.3. Let $z = z'z''$, where $z' = \mathbb{E}Z'$ and $z'' = \mathbb{E}Z''$. Generate R independent replications of each variable, with the two sequences independent. Compare the unbiased estimators

$$\left(\frac{1}{R} \sum_{r=1}^R Z'_r\right) \left(\frac{1}{R} \sum_{r=1}^R Z''_r\right), \quad \frac{1}{R} \sum_{r=1}^R Z'_r Z''_r.$$

Show that the first has no larger variance, and explain why. You may assume Z' and Z'' are independent.



SOLUTION. Write $v' = \text{Var}(Z')$ and $v'' = \text{Var}(Z'')$. Independence gives unbiasedness of both estimators and

$$\text{Var}(\hat{z}_1) = \frac{v'(z'')^2 + v''(z')^2}{R} + \frac{v'v''}{R^2},$$

$$\text{Var}(\hat{z}_2) = \frac{v'(z'')^2 + v''(z')^2 + v'v''}{R}.$$

Therefore,

$$\text{Var}(\hat{z}_2) - \text{Var}(\hat{z}_1) = \frac{R-1}{R^2} v'v'' \geq 0.$$

The product of the two sample means averages over all R^2 cross-pairs, rather than only the R matched pairs. Equality holds if $R = 1$ or either variance is zero. $\square$

EXERCISE 4.4. Give a bivariate central limit theorem for the sample mean $\hat{z}$ and the sample variance s^2 in the i.i.d. setting.

SOLUTION. Assume $\mathbb{E}[(X - \mu)^4] < \infty$, and write $\mu_3 = \mathbb{E}[(X - \mu)^3]$ and $\mu_4 = \mathbb{E}[(X - \mu)^4]$. Then

$$\sqrt{n} \begin{pmatrix} \hat{z} - \mu \\ s^2 - \sigma^2 \end{pmatrix} \rightsquigarrow \mathcal{N} \left(\begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} \sigma^2 & \mu_3 \\ \mu_3 & \mu_4 - \sigma^4 \end{pmatrix} \right).$$

This follows by applying the multivariate CLT to $(X - \mu, (X - \mu)^2 - \sigma^2)$; replacing μ by $\hat{z}$ in the variance contributes only $o_p(n^{-1/2})$. For normal observations, $\mu_3 = 0$ and $\mu_4 = 3\sigma^4$, so the asymptotic variance of s^2 is $2\sigma^4/n$. This special value does not hold for an arbitrary distribution. $\square$

Steady-State Simulation

We now focus on ergodic stochastic processes $Y(\cdot)$. This means that a limiting time-average distribution Y exists (think of a stable queue). Our objective is to estimate

$$z := \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t Y(s) \, ds.$$

Naive approach: let t grow large, and simulate process $Y(\cdot)$ for long time:

$$\hat{z}_T := \frac{1}{T} \int_0^T Y(s) \, ds$$

for *large* T . Note that for small T , the bias effect of an initial transient may be substantial. Under “rather general conditions”, as T grows large,

$$\sqrt{T}(\hat{z}_T - z) \rightarrow \mathcal{N}(0, \sigma^2).$$

This means that we can approximate, with $V \sim \mathcal{N}(0, 1)$,

$$\hat{z}_T \approx z + \frac{\sigma V}{\sqrt{T}},$$

out of which confidence intervals can be constructed. This also entails:

$$\text{Var}(\hat{z}_T) \approx \frac{\sigma^2}{T}.$$

Essentially for the *central limit theorem* to hold, we should have that $\text{Var}\left(\int_0^T Y(s) \, ds\right)$ scales linearly in T as T grows large, with

$$\sigma^2 := \lim_{T \rightarrow \infty} \frac{1}{T} \text{Var}\left(\int_0^T Y(s) \, ds\right)$$

being the proportionality constant.

SECTION 5.1

| Regenerative Method

For regenerative processes, we can define a *regenerative point* T , so that the process “restarts” itself at T . Then the regeneration ratio formula:

$$z = \frac{\mathbb{E}I(T)}{\mathbb{E}T}, \quad I(T) := \int_0^T Y(s) \, ds.$$

For discrete-state-space irreducible ergodic Markov process $X(t)$, one could define a *return state* i^* . Suppose $X(0) = i^*$. Then the r -th return time is defined recursively: $\tau_0 := 0$ and $\tau_r := \inf \{t > \tau_{r-1} : X(t) = i^*, X(t^-) \neq i^*\}$.

$$I_r := \int_{\tau_{r-1}}^{\tau_r} Y(s) ds, \quad T_r = \tau_r - \tau_{r-1}.$$

We simulate R regenerative cycles, providing observations $I_1, \dots, I_R$ and $T_1, \dots, T_R$. Estimate numerator and denominator separately,

$$\hat{z}_R = \frac{\frac{1}{R} \sum_{r=1}^R I_r}{\frac{1}{R} \sum_{r=1}^R T_r} = \frac{\sum_{r=1}^R I_r}{\sum_{r=1}^R T_r}.$$

For the confidence intervals, define $Y_r := I_r - zT_r$, and $Y_r \stackrel{\mathcal{D}}{=} Y$, $I_r \stackrel{\mathcal{D}}{=} I$ and $T_r \stackrel{\mathcal{D}}{=} T$,

$$\eta^2 := \frac{\mathbb{E}[Y^2]}{(\mathbb{E}T)^2}.$$

Then, as $R \rightarrow \infty$,

$$\sqrt{R}(\hat{z}_R - z) \rightarrow \mathcal{N}(0, \eta^2).$$

The confidence interval is given by

$$\left(\hat{z}_R - q_\alpha \frac{\eta}{\sqrt{R}}, \hat{z}_R + q_\alpha \frac{\eta}{\sqrt{R}} \right),$$

after estimating η^2 by

$$\hat{\eta}_R^2 := \frac{\frac{1}{R-1} \sum_{r=1}^R (I_r - \hat{z}_R T_r)^2}{\left(\frac{1}{R} \sum_{r=1}^R T_r \right)^2}.$$

SECTION 5.2

| Batch-Means Method

Cut up the sole observation of the sample path from $[0, t]$ into intervals of length $\frac{t}{R}$. Prerequisite is weak convergence to Brownian motion:

$$\left(\frac{1}{\sqrt{n}} \int_0^{nt} (Y(s) - z) ds \right)_{t \geq 0} \rightarrow (\sigma B(t))_{t \geq 0}.$$

Such a *functional central limit theorem* typically holds when there is only weak dependence in the $Y(\cdot)$ process.

Define contributions of intervals of length $\frac{t}{R}$:

$$\tilde{Y}_r(t) = \frac{1}{\frac{t}{R}} \int_{\frac{(r-1)t}{R}}^{\frac{rt}{R}} Y(s) ds.$$

And there are R of these. Estimator:

$$\hat{z}_R := \frac{1}{R} \sum_{r=1}^R \tilde{Y}_r(t).$$

Note,

$$\mathbb{E}[\hat{z}_R] = \mathbb{E} \left[\frac{1}{R} \sum_{r=1}^R \tilde{Y}_r(t) \right] = \mathbb{E} \left[\frac{1}{t} \int_0^t Y(s) ds \right].$$

Hence, the bias of this estimator decreases as $t \rightarrow \infty$. There is no bias if the process starts in stationarity.

To construct confidence intervals, recall that if $X_1, X_2, \dots, X_R \sim \mathcal{N}(\mu, \sigma^2)$, then for any R ,

$$\sqrt{R} \frac{\frac{1}{R} \sum_{r=1}^R X_r - \mu}{s_R} \sim T_{R-1},$$

where

$$s_R^2 = \frac{1}{R-1} \sum_{r=1}^R \left(X_r - \frac{1}{R} \sum_{s=1}^R X_s \right)^2.$$

Also, due to the prerequisite assumption, we have that the $\tilde{Y}_r(t)$ are asymptotically *i.i.d.* and normally distributed with mean z as $t \rightarrow \infty$. Hence, we plug in $X_r = \tilde{Y}_r(t)$, and we obtain for any R ,

$$\sqrt{R} \frac{\left(\frac{1}{t} \int_0^t Y(s) ds - z \right)}{s_R(t)} \rightarrow T_{R-1}$$

as $t \rightarrow \infty$, and the confidence interval is given by

$$\left(\hat{z}_R - t_{R-1, (1+\alpha)/2} \frac{s_R(t)}{\sqrt{R}}, \hat{z}_R + t_{R-1, (1+\alpha)/2} \frac{s_R(t)}{\sqrt{R}} \right).$$

Here T_R denotes the Student- t distribution with R degrees of freedom, $t_{R,p}$ its p -quantile, and

$$s_R^2(t) = \frac{1}{R-1} \sum_{r=1}^R \left(\frac{1}{\frac{t}{R}} \int_{\frac{(r-1)t}{R}}^{\frac{rt}{R}} Y(s) ds - \frac{1}{t} \int_0^t Y(s) ds \right)^2.$$

Exercises

EXERCISE 5.1. Reconsider the harbor in [Exercise 3.1](#) over a much longer time horizon. We are interested in the long-term average waiting time.

- Discuss how the regenerative method and an adapted simulation program can yield a 95% confidence interval for the average queue length.
- Discuss the corresponding application of batch means.
- Implement either method and report the resulting confidence interval.

SOLUTION. For a regenerative estimate of the mean berth-queue length, let I_r be the integral of the queue length over cycle r and T_r its duration. Use $\sum_r I_r / \sum_r T_r$, with the regenerative variance formula above. Averaging the separate cycle averages gives unequal weights to short and long cycles and is generally incorrect. For batch means, discard an initial warm-up interval, split the remaining time into equal batches, and integrate the berth-queue length exactly between state changes. Apply a Student- t interval to the batch time averages. Sampling the queue only at arrival times would estimate an arrival-weighted quantity.

```
import math
import numpy as np
import simpy
np.random.seed(12345)

def truncated_normal():
    while True:
        u1, u2 = np.random.uniform(size=2)
        z = np.sqrt(-2*np.log(u1))*np.cos(2*np.pi*u2)
        x = 1 + np.sqrt(0.1)*z
        if 0.5 <= x <= 1.5:
            return x

def run_harbor(days, shape=1.5):
    env = simpy.Environment()
    berths = simpy.Resource(env, capacity=2)
    cranes = simpy.Resource(env, capacity=2)
    waits, residence, queue_changes = [], [], [(0.0, 0)]
    scale = 1.25 / math.gamma(1 + 1/shape)

    def record_queue():
        queue_changes.append((env.now, len(berths.queue)))

    def unload():
        arrival = env.now
        with berths.request() as berth:
            record_queue()
            yield berth
            record_queue()
            use_two = berths.count == 1 and not berths.queue and cranes.count
            ↪ == 0
            requests = [cranes.request() for _ in range(2 if use_two else 1)]
            yield env.all_of(requests)
            waits.append(env.now - arrival)
```

```

        yield env.timeout(truncated_normal() / len(requests))
        residence.append(env.now - arrival)
        for request in requests:
            cranes.release(request)

    def arrivals():
        while True:
            u = np.random.uniform()
            yield env.timeout(scale*(-np.log(u))**(1/shape))
            env.process(unload())

    env.process(arrivals())
    env.run(until=days)
    queue_changes.append((days, queue_changes[-1][1]))
    return np.asarray(waits), np.asarray(residence), queue_changes
from scipy.stats import t

warmup, batch_length, number_batches = 1000.0, 5000.0, 20
end = warmup + batch_length*number_batches
_, _, queue_changes = run_harbor(end)
areas = np.zeros(number_batches)
for (left, length), (right, _) in zip(queue_changes, queue_changes[1:]):
    left, right = max(left, warmup), min(right, end)
    while left < right:
        j = min(int((left-warmup)//batch_length), number_batches-1)
        edge = min(right, warmup+(j+1)*batch_length)
        areas[j] += length*(edge-left)
        left = edge
means = areas/batch_length
estimate = np.mean(means)
se = np.std(means, ddof=1)/np.sqrt(number_batches)
half_width = t.ppf(0.975, number_batches-1)*se
print(f"Mean queue length: {estimate:.6f}")
print(f"95% CI: ({estimate-half_width:.6f}, {estimate+half_width:.6f})")

```

```

Mean queue length: 0.011212
95% CI: (0.010354, 0.012071)

```

□

Variance Reduction Methods

SECTION 6.1

| Control Variates

We wish to estimate a performance measure $z := \mathbb{E}[Z]$ using an estimator with smaller variance than the crude Monte Carlo estimator.

DEFINITION 6.1.

The random variable W is called a *control variate for the random variable Z* if Z and W are strongly correlated (either positively or negatively), and $\mathbb{E}[W]$ is known.

We generate replications of (Z, W) and compute the empirical means $\hat{z}$ and $\hat{w}$. Then compute the control variate estimator

$$\hat{z} + \alpha(\hat{w} - \mathbb{E}[W])$$

for an appropriate value of α . Since $\mathbb{E}[\hat{z} + \alpha(\hat{w} - \mathbb{E}[W])] = z$, the estimator is unbiased, and

$$\text{Var}(\hat{z} + \alpha(\hat{w} - \mathbb{E}[W])) = \text{Var}(\hat{z}) + \alpha^2 \text{Var}(\hat{w}) + 2\alpha \text{Cov}(\hat{z}, \hat{w}).$$

We want to minimise the estimator variance, which is a quadratic polynomial, hence optimised for

$$\alpha = -\frac{\text{Cov}(\hat{z}, \hat{w})}{\text{Var}(\hat{w})} = -\frac{\text{Cov}(Z, W)}{\text{Var}(W)}.$$

Using this α , the variance reduces to

$$\begin{aligned} & \frac{\text{Var}(Z)}{R} + \left(-\frac{\text{Cov}(Z, W)}{\text{Var}(W)}\right)^2 \frac{\text{Var}(W)}{R} + 2\left(-\frac{\text{Cov}(Z, W)}{\text{Var}(W)}\right) \frac{\text{Cov}(Z, W)}{R} \\ &= \frac{\text{Var}(Z)}{R} \left(1 - \frac{\text{Cov}(Z, W)^2}{\text{Var}(Z)\text{Var}(W)}\right) = \frac{\text{Var}(Z)}{R} (1 - \rho^2) \end{aligned}$$

where

$$\rho = \text{Corr}(Z, W) = \frac{\text{Cov}(Z, W)}{\sqrt{\text{Var}(Z)\text{Var}(W)}}$$

is the Pearson correlation coefficient between Z and W , which is no larger than $\text{Var}(\hat{z}) = \frac{\text{Var}(Z)}{R}$.

We use unbiased sample estimates for (co-)variances:

$$s_Z^2 = \frac{1}{R-1} \sum_{r=1}^R (Z_r - \hat{z})^2, \quad s_W^2 = \frac{1}{R-1} \sum_{r=1}^R (W_r - \hat{w})^2, \quad s_{ZW}^2 = \frac{1}{R-1} \sum_{r=1}^R (Z_r - \hat{z})(W_r - \hat{w}).$$

Using $\alpha = -\frac{s_{ZW}^2}{s_W^2}$, the variance of the estimator will asymptotically behave as $\frac{\text{Var}(Z)}{R}(1 - \rho^2)$.

Remark. (i) This method is only useful when we can find a control variate W , which has a substantial (linear) correlation with Z .

(ii)

(iii)

When using multiple controls, the single control variate W can be replaced by

$$\mathbf{W} = (W_1, \dots, W_p).$$

The multiple-control estimator is then given by

$$\hat{z} + \boldsymbol{\alpha}^T (\hat{\mathbf{w}} - \mathbb{E}[\mathbf{W}]).$$

EXAMPLE 6.2.

SECTION 6.2

| Antithetic Sampling

We generate *i.i.d.* pairs $(Z_1, Z_2), (Z_3, Z_4), \dots, (Z_{R-1}, Z_R)$, where the Z_i are all distributed like Z is. The pairs are mutually independent, but the Z_i within a pair are dependent.

We then still compute the unbiased estimator

$$\hat{z}_{ant} = \frac{1}{R} \sum_{i=1}^R Z_i.$$

Let's check the variance of the estimator:

$$\begin{aligned} \text{Var}(\hat{z}_{ant}) &= \text{Var}\left(\frac{1}{R} \sum_{i=1}^R Z_i\right) = \frac{1}{2R} \text{Var}(Z_1 + Z_2) \\ &= \frac{1}{2R} (2\text{Var}(Z) + 2\text{Cov}(Z_1, Z_2)) = \frac{\text{Var}(Z)}{R} (1 + \text{Corr}(Z_1, Z_2)). \end{aligned}$$

So, beneficial if $\text{Corr}(Z_1, Z_2)$ negative, and as small as possible.

EXAMPLE 6.3 (Uniform Antithetic Sampling).

THEOREM 6.4.

EXAMPLE 6.5 (Gaussian Antithetic Sampling).

When possible, antithetic sampling is not very hard to implement, but often, the obtained variance reduction is not dramatic.

EXAMPLE 6.6.

SECTION 6.3

| Conditional Monte Carlo

Using classic crude Monte Carlo, we generate *i.i.d.* replicates $Z_1, \dots, Z_R$. Suppose now we would replace Z_i by $\mathbb{E}[Z_i | W_i]$. Thus, our estimator is

$$\frac{1}{R} \sum_{i=1}^R \mathbb{E}[Z_i | W_i] = \mathbb{E} \left[\frac{1}{R} \sum_{i=1}^R Z_i | W \right]$$

where $W = (W_1, \dots, W_R)$ and the pairs (Z_i, W_i) are independent.

By the *law of total expectation*,

$$\mathbb{E} \left[\mathbb{E} \left[\frac{1}{R} \sum_{i=1}^R Z_i | W \right] \right] = \mathbb{E} \left[\frac{1}{R} \sum_{i=1}^R Z_i \right] = \mathbb{E}[Z].$$

We have an unbiased estimator.

THEOREM 6.7 (Law of Total Variance).

For any two random variables U and W with U having finite variance,

$$\text{Var}(U) = \mathbb{E}[\text{Var}(U | W)] + \text{Var}(\mathbb{E}[U | W]).$$

Since variances (and expectations of them) are non-negative, we must have, by taking $U = \frac{1}{R} \sum_{i=1}^R Z_i$, that

$$\text{Var} \left(\frac{1}{R} \sum_{i=1}^R Z_i \right) \geq \text{Var} \left(\mathbb{E} \left[\frac{1}{R} \sum_{i=1}^R Z_i | W \right] \right).$$

EXAMPLE 6.8. Choosing W is very much situation-dependent. Consider one can estimate π by computing $\frac{1}{R} \sum_{i=1}^R Z_i$ where $Z_i = 4\mathbb{1}_{\{(u_1^{(i)})^2 + (u_2^{(i)})^2 \leq 1\}}$ and $U_1^{(i)}, U_2^{(i)}$ are *i.i.d.* uniform on $[0, 1]$. And we could condition on (the samples of) U_1 :

$$\begin{aligned} \mathbb{E} \left[\frac{1}{R} \sum_{i=1}^R Z_i | U_1^{(1)}, \dots, U_1^{(R)} \right] &= \frac{4}{R} \sum_{i=1}^R \mathbb{P} \left((U_1^{(i)})^2 + (U_2^{(i)})^2 \leq 1 | U_1^{(i)} \right) \\ &= \frac{4}{R} \sum_{i=1}^R \mathbb{P} \left(U_2^{(i)} \leq \sqrt{1 - (U_1^{(i)})^2} | U_1^{(i)} \right) = \frac{4}{R} \sum_{i=1}^R \sqrt{1 - (U_1^{(i)})^2}. \end{aligned}$$

Therefore, we would simply generate R uniform samples on $[0, 1]$ and compute

$$\frac{4}{R} \sum_{i=1}^R \sqrt{1 - (U_1^{(i)})^2}$$

. We only need half the uniform samples, and less sampling in this case means less variance: more than three times as small.

EXAMPLE 6.9.

SECTION 6.4

| Splitting

We want to estimate the expectation of a random variable $Z = \varphi(X, Y)$. Suppose that samples from X are computationally much harder to generate than samples from Y , and the value of $\varphi(X, Y)$ is much more influenced by Y than by X .

The idea is to simply reuse samples of X a number of S times. We use the splitting estimator

$$\frac{1}{RS} \sum_{r=1}^R \sum_{s=1}^S \varphi(X_r, Y_{rs}),$$

which needs fewer replicates of X , so far less computation time. And about the same variance because $\text{Corr}(\varphi(X_r, Y_{r1}), \varphi(X_r, Y_{r2}))$ is close to zero as $\varphi(X, Y)$ is much more influenced by Y .

In deciding how to choose S , we cannot just compare variances anymore, since computation time now plays a role. Let's say that a is the time to generate a sample of X , b a sample of Y , $a \gg b$.

$$Z = \varphi(X, Y), \quad \tilde{Z} = \frac{1}{S} \sum_{s=1}^S \varphi(X, Y_s).$$

The time to get one estimate of Z approximately equals $a + b$, and the time to get one estimate of $\tilde{Z}$ approximately equals $a + Sb$. In a given time t , one gets $\frac{t}{a+b}$ estimates of Z , and $\frac{t}{a+Sb}$ estimates of $\tilde{Z}$. Recall that the width of a confidence interval of an estimate for z is generally given by:

$$\left(z - q_\alpha \frac{\sigma}{\sqrt{R}}, z + q_\alpha \frac{\sigma}{\sqrt{R}} \right),$$

where R is the number of estimates. Both Z and $\tilde{Z}$ are unbiased estimators, so we wish to minimise $\frac{\sigma}{\sqrt{R}}$. We need to compare $e := (a + b)\text{Var}(Z)$ (corresponding to one iterate of crude Monte Carlo) with $\tilde{e} := (a + Sb)\text{Var}(\tilde{Z})$ (one "splitting" iterate). Suppose that $\text{Var}(Z) = 1$ and $\rho = \text{Corr}(\varphi(X_r, Y_{r1}), \varphi(X_r, Y_{r2}))$. Then

$$\begin{aligned} \text{Var}(\tilde{Z}) &= \frac{1}{S^2} \left(\text{Cov} \left(\sum_{s=1}^S \varphi(X, Y_s), \sum_{s=1}^S \varphi(X, Y_s) \right) \right) \\ &= \frac{1}{S^2} \left(\sum_{r=1}^S \sum_{s=1}^S \text{Cov}(\varphi(X, Y_r), \varphi(X, Y_s)) \right) = \frac{1}{S^2} (S + (S^2 - S)\rho), \end{aligned}$$

where for S terms that $r = s$, we are calculating $\sum_{s=1}^S \text{Var}(\varphi(X, Y_s)) = S$; for $S(S - 1)$ terms that $r \neq s$, each term contributes ρ . Therefore,

$$e = a + b, \quad \tilde{e} = (a + Sb) \left(\frac{1}{S} + \left(1 - \frac{1}{S} \right) \rho \right).$$

For large S , the $\tilde{e}$ -approximation $(a + Sb)(\frac{1}{S} + \rho)$ is minimised by $S = \sqrt{\frac{a}{\rho b}}$, so that the optimal efficiency is approximated by $a\left(\sqrt{\frac{b}{a}} + \sqrt{\rho}\right)^2$. This leads to

$$\frac{\tilde{e}}{e} = \frac{a}{a+b} \left(\sqrt{\frac{b}{a}} + \sqrt{\rho} \right)^2.$$

In conclusion, with the right choice of S , and small values of $\frac{b}{a}$ and ρ , splitting nets you the desired precision faster.

EXAMPLE 6.10 (Discrete Event Simulation). A restaurant opens at 10.00; customers spend an exponentially distributed amount of time dining. Not many arrivals until noon, but especially between 12.30 and 13.30 there are many, many arrivals. We want to estimate the expected number of lost customers between noon and 14.00 due to restaurant being full. How to take X and Y ?

Number of lost customers more influenced by arrivals after noon than by number of occupied tables at noon. Arrivals after noon not correlated with occupancy at noon. Then choose X to be number of occupied tables at noon, along with the residual service times. Choose Y to be the arrival times and service times of customers arriving between noon and 14.00.

EXAMPLE 6.11. Suppose you want to estimate the expectation of a functional $Z = Z(B)$, where B is a standard Brownian motion in $[0, 1]$.

We use 16 grid regions. Generate $B(\frac{1}{16}), B(\frac{2}{16}), \dots, B(1)$ by summing 16 *i.i.d.* normal samples. Replace linear interpolations by Brownian bridges $W_{i,c}(t)$. These are Brownian motions constrained not only by $W(0) = 0$, but also by a value $W(\frac{1}{16}) = c$, where c is the difference between the sampled B -points:

$$W_i(t) = B_i(t) + 16t \left(c - B_i\left(\frac{1}{16}\right) \right),$$

where the B_i are independent Brownian motions. These can be sampled, each using $\frac{1024}{16} = 64$ grid regions. Choose X to be 16 Brownian bridges. These do not have to be sampled often. Choose Y to be the numbers $B(\frac{1}{16}), B(\frac{2}{16}), \dots, B(1)$.

SECTION 6.5

| Common Random Numbers

We want to compute the difference between the means z', z'' of two random variables Z' and Z'' with $\text{Var}(Z') = \text{Var}(Z'') = \sigma^2$ that behave in some sense similar to the random input $U_1, \dots, U_n$.

We could just use $U_1, \dots, U_n$ for the estimate of Z'' as well. Thus, common random numbers advocates replicating

$$Z'(U_1, \dots, U_n) - Z''(U_1, \dots, U_n)$$

R times and computing the sample mean. This estimator has variance

$$\frac{2\sigma^2 - 2\text{Cov}(Z'(U_1, \dots, U_n), Z''(U_1, \dots, U_n))}{R}.$$

Since Z' and Z'' behave similar to the random input, which yields positive, significant covariance, we have high variance reduction.

Remark. (i) This method needs fewer input samples (the $U_1, \dots, U_n$), improving computation time.

(ii)

(iii)

EXAMPLE 6.12.

SECTION 6.6

| Stratification

Suppose we want to estimate $\mathbb{E}[Z]$, where $Z = g(X)$. The idea behind the method of *stratification* is to partition the sample space Ω into regions $\Omega_1, \dots, \Omega_S$ called *strata*. The aim is to eliminate as much of the variation *between strata* as possible.

The strata are often obtained by dividing the range of one or more important random variables driving the simulation. For example, when $Z = g(X)$, where X is standard uniformly distributed, we could partition the support $[0, 1]$ of X into the strata

$$\Omega_s = \left\{ \frac{s-1}{S} \leq X \leq \frac{s}{S} \right\}.$$

Moreover, when Z_s is a random variable having the distribution of Z conditioned on Ω_s , we have

$$\mathbb{P}(Z_s \in A) = \mathbb{P}(Z \in A \mid \Omega_s) = \frac{\mathbb{P}(Z \in A, \Omega_s)}{\mathbb{P}(\Omega_s)}.$$

Strategy. Divide the total number of R replicates into $R_1, \dots, R_S$. For each $s \in \{1, \dots, S\}$, simulate R_s replicates of Z_s . Estimate z_s by the empirical average $\hat{z}_s = \frac{1}{R_s} \sum_{j=1}^{R_s} Z_j$. Compute the following estimate for $\mathbb{E}[Z]$:

$$\hat{z}_{str} = \sum_{s=1}^S p_s \hat{z}_s,$$

where $p_s = \mathbb{P}(\Omega_s)$.

Note that

$$\begin{aligned} \mathbb{E}[\hat{z}_{str}] &= \mathbb{E}\left[\sum_{s=1}^S p_s \hat{z}_s\right] = \mathbb{E}\left[\sum_{s=1}^S p_s \frac{1}{R_s} \sum_{j=1}^{R_s} Z_j\right] = \sum_{s=1}^S p_s \frac{1}{R_s} \sum_{j=1}^{R_s} \mathbb{E}[Z \mid \Omega_s] \\ &= \sum_{s=1}^S p_s \mathbb{E}[Z \mid \Omega_s] = \mathbb{E}[Z]. \end{aligned}$$

Hence, the estimator is unbiased.

For the variance, we have

$$\text{Var}(\hat{z}_{str}) = \text{Var}\left(\sum_{s=1}^S p_s \hat{z}_s\right) = \sum_{s=1}^S p_s^2 \text{Var}(\hat{z}_s) = \sum_{s=1}^S \frac{p_s^2 \sigma_s^2}{R_s}.$$

Whether $\text{Var}(\hat{z}_{str}) < \text{Var}(\hat{z})$ depends on the actual choice of R_s . We could adopt *proportional allocation*, i.e., take R_s such that $p_s = \frac{R_s}{R}$. Then

$$\text{Var}(\hat{z}_{str}) = \sum_{s=1}^S \frac{p_s \sigma_s^2}{R}.$$

Note that $\sum_{s=1}^S p_s \sigma_s^2$ can be interpreted as $\mathbb{E}[\text{Var}(Z_J \mid J)]$, where J satisfied $\mathbb{P}(J = s) = p_s$ for $s = 1, \dots, S$, so that

$$\text{Var}(\hat{z}_{str}) = \frac{1}{R} \mathbb{E}[\text{Var}(Z_J \mid J)] \leq \frac{1}{R} (\mathbb{E}[\text{Var}(Z_J \mid J)] + \text{Var}(\mathbb{E}[Z_J \mid J])) = \frac{1}{R} \text{Var}(Z_J) = \text{Var}(\hat{z}).$$

Thus, there is always variance reduction.

To evaluate the confidence interval, we have

$$\sqrt{R_s}(\hat{z}_s - z_s) \rightarrow \mathcal{N}(0, \sigma_s^2)$$

as $R_s \rightarrow \infty$, where $\sigma_s^2 = \text{Var}(Z_s) = \text{Var}(Z \mid \Omega_s)$. Hence,

$$\sqrt{R}(\hat{z}_{str} - z) \rightarrow \mathcal{N}\left(0, \sum_{s=1}^S \frac{p_s^2 \sigma_s^2}{R_s/R}\right)$$

as $R \rightarrow \infty$ in such a way that the R_s/R have non-zero limits.

EXAMPLE 6.13. Suppose again that $Z = g(X)$, where X is a standard uniform random variable, and we choose the strata

$$\Omega_s = \left\{ \frac{s-1}{S} \leq X \leq \frac{s}{S} \right\}.$$

EXAMPLE 6.14.

A variant of proportional allocation R_s is *poststratification*: allocate the R_s on-the-fly during the simulation, so that $R_s \approx R p_s$. Let the simulation generate *i.i.d.* samples (Σ_r, Z_r) where $r = 1, \dots, R$ from a multi-variate random variable (Σ, Z) such that $\mathbb{P}(\Sigma = s) = p_s$ and Z conditioned on $\Sigma = s$ has the same distribution as Z_s . Then, we estimate $\mathbb{E}[Z_s]$ by the empirical average

$$\hat{z}_s = \frac{1}{R_s} \sum_{r: \Sigma_r = s} Z_r.$$

Then, we can use

$$\hat{z}_{poststr} = \sum_{s=1}^S p_s \hat{z}_s.$$

Remark. (i) In the limit, the same central limit theorem as for proportional allocation holds.

(ii) Usable when hard to sample directly from Z_s , but knowledge on the p_s can be used.

(iii) Still quite larger variance for small R than proportional allocation, as the R_s are random variables themselves adding to the variance.

EXAMPLE 6.15 (Latin Hypercube Sampling). We want to sample R d -dimensional uniform random variables $\mathbf{V}_1, \dots, \mathbf{V}_R \in [0, 1]^d$.

SECTION 6.7

| Indirect Sampling

In some cases, some parts of the expectation $\mathbb{E}[Z]$ can be evaluated analytically. Sometimes, $\mathbb{E}[Z]$ may be related to the expectations of some other random variables. When such phenomena may be exploited for variance reduction, this is called *indirect estimation*.

EXAMPLE 6.16.

SECTION 6.8

| Importance Sampling

Suppose we want to estimate some performance measure $z = \mathbb{E}[Z] = \mathbb{E}[h(\mathbf{X})]$. We allow the output variable $h(\mathbf{X})$ to be a function of a random input vector $\mathbf{X} = (X_1, \dots, X_n)$. For ease of discussion, let's say that $\mathbf{X}$ has a joint density (or mass) function $f(\mathbf{x}) = f(x_1, \dots, x_n)$ and/or CDF $F(\mathbf{x})$. Thus

$$z = \mathbb{E}[h(\mathbf{X})] = \int h(\mathbf{x})f(\mathbf{x}) \, d\mathbf{x}.$$

Suppose now that a crude Monte Carlo simulation of z is inefficient, because it is difficult to sample a random vector having density function $f(\mathbf{x})$, and/or the variance of $h(\mathbf{X})$ is just very large, requiring too many replicates. In either of these cases, *importance sampling* might be a lifesaver.

Instead of generating replicates of $h(\mathbf{X})$ and averaging, we note that

$$z = \int h(\mathbf{x}) \frac{f(\mathbf{x})}{g(\mathbf{x})} g(\mathbf{x}) \, d\mathbf{x} = \mathbb{E}_g \left[h(\mathbf{X}) \frac{f(\mathbf{X})}{g(\mathbf{X})} \right] = \mathbb{E}_g [h(\mathbf{X})L(\mathbf{X})],$$

where $L(\mathbf{x}) = \frac{f(\mathbf{x})}{g(\mathbf{x})}$ is called a *likelihood ratio function*. This suggests the following estimation scheme: create R replicates of $\mathbf{X}, \mathbf{X}_1, \dots, \mathbf{X}_R$ as if they have density $g(\mathbf{x})$. Compute the values $h(\mathbf{X}_i)L(\mathbf{X}_i)$ for $i = 1, \dots, R$, then average these values to get an importance sampling estimate based on importance sampling density g .

Remark. This method works only if $g(\mathbf{x}) = 0$ implies $h(\mathbf{x})f(\mathbf{x}) = 0$.

EXAMPLE 6.17. (i) If X is $\text{Bin}(n, p)$ distributed,

$$\mathbb{E}[h(X)]$$

(ii) If X is $\exp(\lambda)$ distributed,

$$\mathbb{E}[h(X)] = \int_{x=0}^{\infty} h(x) \lambda e^{-\lambda x} \, dx = \int_{x=0}^{\infty} h(x) \frac{\lambda e^{-\lambda x}}{\mu e^{-\mu x}} \mu e^{-\mu x} \, dx = \mathbb{E}_g \left[h(X) \frac{\lambda e^{-\lambda X}}{\mu e^{-\mu X}} \right],$$

where X is $\exp(\mu)$ distributed under density g .

Our importance sampling estimator now is

$$\hat{z}_{IS} = \frac{1}{R} \sum_{i=1}^R h(\mathbf{X}_i)L(\mathbf{X}_i)$$

with $\mathbf{X}_i$ sampled from density g . We already saw that

$$\mathbb{E}_g[\hat{z}_{IS}] = \mathbb{E}_g[h(\mathbf{X})L(\mathbf{X})] = z,$$

which implies an unbiased estimator, regardless of the choice of g . For the variance, we have

$$\begin{aligned} \sigma_{IS}^2 &= \text{Var}_g \left(\frac{1}{R} \sum_{i=1}^R h(\mathbf{X}_i)L(\mathbf{X}_i) \right) = \frac{1}{R} \text{Var}_g(h(\mathbf{X})L(\mathbf{X})) \\ &= \frac{1}{R} \left(\mathbb{E}_g[h^2(\mathbf{X})L^2(\mathbf{X})] - \mathbb{E}[h(\mathbf{X})]^2 \right) = \frac{1}{R} \left(\mathbb{E}[h^2(\mathbf{X})L(\mathbf{X})] - \mathbb{E}[h(\mathbf{X})]^2 \right). \end{aligned}$$

This number can be estimated by the sample variance as usual, and confidence intervals can be built. Problem:

$$\min_g \left\{ \mathbb{E}_g [h^2(\mathbf{X})L^2(\mathbf{X})] : L(\mathbf{X}) = \frac{f(\mathbf{X})}{g(\mathbf{X})} \right\}.$$

THEOREM 6.18.

An often used successful *change of measure* is given by the technique of *exponential tilting*, where we choose

$$g_\theta(\mathbf{x}) = e^{\theta^T \mathbf{x} - \kappa(\theta)} f(\mathbf{x}),$$

where $\kappa(\theta) = \log \mathbb{E} [e^{\theta^T \mathbf{X}}]$. For one-dimensional densities, this implies

$$g_\theta(x) = \frac{e^{\theta x}}{\mathbb{E}[e^{\theta X}]} f(x).$$

EXAMPLE 6.19. (i) The exponential distribution $f(x) = \lambda e^{-\lambda x}$, where $f_\theta(x) = \lambda_\theta e^{-\lambda_\theta x}$ for $\lambda_\theta = \lambda - \theta$, $-\infty < \theta < \lambda$. More generally, for the Gamma density with parameters α and λ , i.e.,

$$f(x) = \frac{\lambda^\alpha x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)},$$

one gets

$$f_\theta(x) = \frac{\lambda_\theta^\alpha x^{\alpha-1} e^{-\lambda_\theta x}}{\Gamma(\alpha)},$$

where again $\lambda_\theta = \lambda - \theta$.

- (ii) The $\mathcal{N}(\mu, \sigma^2)$ distribution gets tilted into the $\mathcal{N}(\mu + \theta\sigma^2, \sigma^2)$.
- (iii) The $\text{Bin}(n, \alpha)$ distribution

$$p(k) = \binom{n}{k} \alpha^k (1 - \alpha)^{n-k}, \quad k = 0, \dots, n$$

gets tilted into the $\text{Bin}(n, \alpha_\theta)$ with

$$\alpha_\theta = \frac{\alpha e^\theta}{1 - \alpha + \alpha e^\theta}, \quad \theta \in \mathbb{R}.$$

- (iv) The $\text{Poisson}(\mu)$ distribution $p(k) = \frac{e^{-\mu} \mu^k}{k!}$ gets tilted into the $\text{Poisson}(\mu e^\theta)$ distribution, $\theta \in \mathbb{R}$.
- (v) Multivariate: the $\mathcal{N}(\boldsymbol{\mu}, \mathbf{C})$ distribution gets tilted into the $\mathcal{N}(\boldsymbol{\mu} + \mathbf{C}\boldsymbol{\theta}, \mathbf{C})$ distribution.

Proof. (i) Applying the tilting measure to $f(x) = \lambda e^{-\lambda x}$, we have

$$f_\theta(x) = e^{\theta x - \kappa(\theta)} f(x),$$

where $\kappa(\theta) = \log \mathbb{E} [e^{\theta X}]$. Then we calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \int_0^\infty e^{\theta x} \lambda e^{-\lambda x} dx = \log \int_0^\infty \lambda e^{-(\lambda - \theta)x} dx = \log \left(\frac{\lambda}{\lambda - \theta} \right).$$

Therefore,

$$f_\theta(x) = e^{\theta x - \kappa(\theta)} f(x) = e^{\theta x - \log(\frac{\lambda}{\lambda - \theta})} \lambda e^{-\lambda x} = \frac{\lambda - \theta}{\lambda} e^{\theta x} \lambda e^{-\lambda x} = \lambda_\theta e^{-\lambda_\theta x},$$

where $\lambda_\theta = \lambda - \theta$, and $-\infty < \theta < \lambda$.

Similarly, for the Gamma density

$$f(x) = \frac{\lambda^\alpha x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)},$$

we first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \mathbb{E}[e^{\theta X}] = \log \int_0^\infty e^{\theta x} \frac{\lambda^\alpha x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} dx = \alpha \log \left(\frac{\lambda}{\lambda - \theta} \right).$$

Then applying the tilting measure, we have

$$f_\theta(x) = e^{\theta x - \alpha \log \left(\frac{\lambda}{\lambda - \theta} \right)} \frac{\lambda^\alpha x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} = \left(\frac{\lambda - \theta}{\lambda} \right)^\alpha \frac{e^{\theta x} \lambda^\alpha x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} = \frac{\lambda_\theta^\alpha x^{\alpha-1} e^{-\lambda_\theta x}}{\Gamma(\alpha)},$$

where again $\lambda_\theta = \lambda - \theta$.

(ii)

(iii)

(iv)

(v)

□

EXAMPLE 6.20. Suppose $X_1, \dots, X_n$ are *i.i.d.* with common density $f(x)$ and suppose that the importance distribution preserves the *i.i.d.* property but changes $f(x)$ to $g_\theta(x) = \frac{e^{\theta x}}{\mathbb{E}[e^{\theta X}]} f(x)$.

Then, note that

$$L(\mathbf{X}) = \frac{f^n(\mathbf{X})}{g_\theta^n(\mathbf{X})} = \prod_{i=1}^n \frac{f(X_i)}{g_\theta(X_i)} = e^{-\theta S_n} \left(\mathbb{E}[e^{\theta X_1}] \right)^n,$$

where $S_n = \sum_{i=1}^n X_i$. This is a very simple form.

Remark. (i) Exponential tilting is often used in problems involving light-tailed random variables (i.e. random variables where tails of distribution decay exponentially fast). In this case, for instance, it can be used to make large values of S_n more likely.

(ii) If we aim for values of t or larger, a common choice is to choose θ such that $\mathbb{E}_{g_\theta}[S_n] = t$.

EXAMPLE 6.21 (Stopped Processes).

EXAMPLE 6.22 (Markov Chains). (i) Let $\{X_s : s \in \mathbb{N}\}$ be a time-inhomogeneous discrete-time Markov chain on a countable state space.

(ii) Let $\{X(t), t \geq 0\}$ be a time-homogeneous continuous-time Markov chain on a countable state space.

EXAMPLE 6.23 (Compound Poisson Processes).

Exercises

EXERCISE 6.1. Let X be bivariate normal with mean zero and covariance

$$C = \begin{pmatrix} 4 & -1 \\ -1 & 4 \end{pmatrix}, \quad A = \{(x, y) : x \geq a, y \geq a\}.$$

For $a = 1, 3, 10$:

- (i) Estimate $z = \mathbb{P}(X \in A)$ and give a 95% confidence interval using crude Monte Carlo.
- (ii) Find $b \in A$ maximizing the $N(0, C)$ density and repeat using importance sampling from $N(b, C)$.
- (iii) Experiment with $N(b, \delta C)$ for different admissible values of δ .

SOLUTION. (i) We generate a large number of 2-dimensional Gaussian random vector $\mathbf{X} = (x, y)$ with mean $\mathbf{0}$ and covariance $\mathbf{C} = \begin{pmatrix} 4 & -1 \\ -1 & 4 \end{pmatrix}$. For each $\mathbf{X}$, we check whether it falls into $A = \{(x, y) : x \geq a, y \geq a\}$ by checking if both $x \geq a$ and $y \geq a$. Then the estimation of z is

$$\hat{z} = \frac{N_A}{N},$$

where N_A is the number of samples that fall in A , and N is the total number of samples. We can calculate the 95% confidence interval by

$$\left[B_{N_A, N-N_A+1}^{-1}(0.025), B_{N_A+1, N-N_A}^{-1}(0.975) \right].$$

Here $B_{a,b}^{-1}$ is the Beta quantile function; the lower endpoint is zero if $N_A = 0$, and the upper endpoint is one if $N_A = N$. The following code implements this exact binomial interval.

```
import numpy as np
np.random.seed(12345)

def box_muller_transform(uniform_samples):
    u1, u2 = uniform_samples[:, 0], uniform_samples[:, 1]
    z0 = np.sqrt(-2 * np.log(u1)) * np.cos(2 * np.pi * u2)
    z1 = np.sqrt(-2 * np.log(u1)) * np.sin(2 * np.pi * u2)
    return np.column_stack((z0, z1))

def gaussian(mean, covariance, N):
    uniform_samples = np.random.rand(N, 2)
    standard_normal_samples = box_muller_transform(uniform_samples)
    A = np.linalg.cholesky(covariance)
    gaussian_samples = mean + np.dot(standard_normal_samples, A.T)
    return gaussian_samples

def simulation(a, N = 10000):
    mean = np.array([0, 0])
    covariance = np.array([[4, -1], [-1, 4]])

    samples = gaussian(mean, covariance, N)
    z_hat = np.sum((samples[:, 0] >= a) & (samples[:, 1] >= a)) / N
```

```

from scipy.stats import beta
hits = int(round(z_hat * N))
lo = 0.0 if hits == 0 else beta.ppf(0.025, hits, N-hits+1)
hi = 1.0 if hits == N else beta.ppf(0.975, hits+1, N-hits)
confidence_interval = (lo, hi)

return z_hat, confidence_interval

for a in [1, 3, 10]:
    z_hat, confidence_interval = simulation(a)
    print(f"For a = {a}, z = {z_hat}, 95% Confidence Interval:
    ↪ {confidence_interval}")

```

Notice that we have used Box-Muller transform to generate independent standard normal samples Z_0, Z_1 from independent standard uniform samples U_1, U_2 , and then use Cholesky decomposition to generate Gaussian samples. An output of this code is as follows.

```

For a = 1, z = 0.0636, 95% Confidence Interval: (0.05889455539681176,
↪ 0.0685615068771029)
For a = 3, z = 0.0013, 95% Confidence Interval: (0.0006923710885752354,
↪ 0.0022220136212590122)
For a = 10, z = 0.0, 95% Confidence Interval: (0.0, 0.0003688199146187622)

```

- (ii) For Gaussian distribution, the density is highest at the mean.

Given $\mathbf{b} \in A = \{(x, y) : x \geq a, y \geq a\}$, we let $\mathbf{b} = (a, a)$ closest to the mean to be the maximising point. In the case, for each sample $\mathbf{X}_i$, we calculate the variance of the weighted estimates as

$$\text{Var} = \frac{1}{N} \sum_{i=1}^N (L_i \mathbb{1}_{\{\mathbf{X}_i \in A\}})^2 - \hat{z}^2,$$

where $i = 1, 2, \dots, N$, and $\hat{z}$ is the mean of these weighted estimates. Then the standard error is $\sqrt{\frac{\text{Var}}{N}}$. The following is the code that implements the above process.

```

import numpy as np
np.random.seed(12345)

def box_muller_transform(uniform_samples):
    u1, u2 = uniform_samples[:, 0], uniform_samples[:, 1]
    z0 = np.sqrt(-2 * np.log(u1)) * np.cos(2 * np.pi * u2)
    z1 = np.sqrt(-2 * np.log(u1)) * np.sin(2 * np.pi * u2)
    return np.column_stack((z0, z1))

def gaussian(mean, covariance, N):
    uniform_samples = np.random.rand(N, 2)
    standard_normal_samples = box_muller_transform(uniform_samples)
    A = np.linalg.cholesky(covariance)
    gaussian_samples = mean + np.dot(standard_normal_samples, A.T)
    return gaussian_samples

def importance_sampling(a, N = 10000):

```

```

mean = np.array([0, 0])
covariance = np.array([[4, -1], [-1, 4]])
b = np.array([a, a])
C_inv = np.linalg.inv(covariance)

importance_samples = gaussian(b, covariance, N)
L = np.exp(-np.dot(np.dot(b.T, C_inv), importance_samples.T) + 0.5 *
    ↪ np.dot(np.dot(b.T, C_inv), b))
indicator = (importance_samples[:, 0] >= a) & (importance_samples[:,
    ↪ 1] >= a)
weighted_samples = L * indicator
z_hat = np.mean(weighted_samples)
variance = np.mean(weighted_samples**2) - z_hat**2

std_error = np.sqrt(variance / N)
confidence_interval = (z_hat - 1.96 * std_error, z_hat + 1.96 *
    ↪ std_error)

return z_hat, confidence_interval

for a in [1, 3, 10]:
    z_hat, confidence_interval = importance_sampling(a)
    print(f"For a = {a}, z = {z_hat}, 95% Confidence Interval:
    ↪ {confidence_interval}")

```

An output of this code is as follows.

```

For a = 1, z = 0.06541763373337597, 95% Confidence Interval:
    ↪ (0.06262603288294509, 0.06820923458380686)
For a = 3, z = 0.001408439272518163, 95% Confidence Interval:
    ↪ (0.0013199499134633856, 0.0014969286315729404)
For a = 10, z = 1.2556825889130994e-17, 95% Confidence Interval:
    ↪ (1.0557483186733641e-17, 1.4556168591528348e-17)

```

(iii) We modify the previous codes and test $\delta = 0.5, 1, 2, 5, 20, 100$.

```

import numpy as np
np.random.seed(12345)

def box_muller_transform(uniform_samples):
    u1, u2 = uniform_samples[:, 0], uniform_samples[:, 1]
    z0 = np.sqrt(-2 * np.log(u1)) * np.cos(2 * np.pi * u2)
    z1 = np.sqrt(-2 * np.log(u1)) * np.sin(2 * np.pi * u2)
    return np.column_stack((z0, z1))

def gaussian(mean, covariance, N):
    uniform_samples = np.random.rand(N, 2)
    standard_normal_samples = box_muller_transform(uniform_samples)
    A = np.linalg.cholesky(covariance)
    gaussian_samples = mean + np.dot(standard_normal_samples, A.T)
    return gaussian_samples

```

```

def importance_sampling(a, delta, N=10000):
    mean = np.array([0, 0])
    covariance = np.array([[4, -1], [-1, 4]])
    scaled_covariance = delta * covariance
    b = np.array([a, a])
    C_inv = np.linalg.inv(covariance)

    importance_samples = gaussian(b, scaled_covariance, N)
    centered = importance_samples - b
    q_original = np.einsum("ni,ij,nj->n", importance_samples, C_inv,
        ↪ importance_samples)
    q_proposal = np.einsum("ni,ij,nj->n", centered, C_inv, centered) /
        ↪ delta
    L = delta * np.exp(0.5 * (q_proposal - q_original))
    indicator = (importance_samples[:, 0] >= a) & (importance_samples[:,
        ↪ 1] >= a)
    weighted_samples = L * indicator
    z_hat = np.mean(weighted_samples)
    variance = np.mean(weighted_samples**2) - z_hat**2

    std_error = np.sqrt(variance / N)
    confidence_interval = (z_hat - 1.96 * std_error, z_hat + 1.96 *
        ↪ std_error)

    return z_hat, confidence_interval

delta_values = [0.5, 1, 2, 5, 20, 100]

for delta in delta_values:
    print(f"Delta = {delta}")
    for a in [1, 3, 10]:
        z_hat, confidence_interval = importance_sampling(a, delta)
        print(f" For a = {a}, z = {z_hat}, 95% Confidence Interval:
            ↪ {confidence_interval}")

```

And an output of this code is as follows.

```

Delta = 0.5
For a = 1, z = 0.06462381351649914, 95% Confidence Interval:
↪ (0.06201123048399774, 0.06723639654900053)
For a = 3, z = 0.001403116612188684, 95% Confidence Interval:
↪ (0.0013382315479431904, 0.0014680016764341774)
For a = 10, z = 1.2208917405227898e-17, 95% Confidence Interval:
↪ (1.0770987689054382e-17, 1.3646847121401414e-17)
Delta = 1
For a = 1, z = 0.06611497039908412, 95% Confidence Interval:
↪ (0.0633454616192266, 0.06888447917894164)
For a = 3, z = 0.001415642695886623, 95% Confidence Interval:
↪ (0.0013230642236697087, 0.0015082211681035373)
For a = 10, z = 1.1411759419180781e-17, 95% Confidence Interval:
↪ (9.447522658260138e-18, 1.3375996180101424e-17)

```

```

Delta = 2
For a = 1, z = 0.0638020413244798, 95% Confidence Interval:
  ↪ (0.060099627523319696, 0.06750445512563989)
For a = 3, z = 0.0013486677628460778, 95% Confidence Interval:
  ↪ (0.0012286463855546554, 0.0014686891401375001)
For a = 10, z = 1.3586690731728724e-17, 95% Confidence Interval:
  ↪ (1.0262654062393153e-17, 1.6910727401064293e-17)
Delta = 5
For a = 1, z = 0.06688297802991268, 95% Confidence Interval:
  ↪ (0.06088359712025629, 0.07288235893956907)
For a = 3, z = 0.0013276558492888938, 95% Confidence Interval:
  ↪ (0.0011453185736816876, 0.0015099931248961)
For a = 10, z = 1.2256850109441169e-17, 95% Confidence Interval:
  ↪ (7.927590111536407e-18, 1.658611010734593e-17)
Delta = 20
For a = 1, z = 0.055170458238996925, 95% Confidence Interval:
  ↪ (0.04442385908669119, 0.06591705739130266)
For a = 3, z = 0.0015117791119690295, 95% Confidence Interval:
  ↪ (0.0011219594747606133, 0.0019015987491774458)
For a = 10, z = 1.7073512901321444e-17, 95% Confidence Interval:
  ↪ (7.194069191249694e-18, 2.6952956611393195e-17)
Delta = 100
For a = 1, z = 0.08903121289530656, 95% Confidence Interval:
  ↪ (0.058219445896953076, 0.11984297989366005)
For a = 3, z = 0.0018487385316050802, 95% Confidence Interval:
  ↪ (0.0006533636922047698, 0.0030441133710053907)
For a = 10, z = 6.468937517116499e-18, 95% Confidence Interval:
  ↪ (5.214726049448319e-19, 1.2416402429288166e-17)

```

□

EXERCISE 6.2. Verify the exponential-tilting claims in Section V.1b of [1]. With $\kappa(\theta) = \log \mathbb{E}e^{\theta X}$ (or $\log \mathbb{E}e^{\theta^T X}$ in the vector case), show that the tilt proportional to $e^{\theta X}$ gives:

- (i) Exponential($\lambda - \theta$) from Exponential(λ), for $\theta < \lambda$; more generally Gamma($\alpha, \lambda - \theta$) from Gamma(α, λ), with rate parametrization.
- (ii) Binomial(N, α_θ) from Binomial(N, α), where $\alpha_\theta = \alpha e^\theta / (1 - \alpha + \alpha e^\theta)$.
- (iii) Poisson(μe^θ) from Poisson(μ).
- (iv) $N(\mu + \theta \sigma^2, \sigma^2)$ from $N(\mu, \sigma^2)$.
- (v) $N(\mu + C\theta, C)$ from $N(\mu, C)$.

SOLUTION. (i) Applying the tilting measure to $f(x) = \lambda e^{-\lambda x}$, we have

$$f_\theta(x) = e^{\theta x - \kappa(\theta)} f(x),$$

where $\kappa(\theta) = \log \mathbb{E}[e^{\theta X}]$. Then we calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \int_0^\infty e^{\theta x} \lambda e^{-\lambda x} dx = \log \int_0^\infty \lambda e^{-(\lambda - \theta)x} dx = \log \left(\frac{\lambda}{\lambda - \theta} \right).$$

Therefore,

$$f_{\theta}(x) = e^{\theta x - \kappa(\theta)} f(x) = e^{\theta x - \log\left(\frac{\lambda}{\lambda - \theta}\right)} \lambda e^{-\lambda x} = \frac{\lambda - \theta}{\lambda} e^{\theta x} \lambda e^{-\lambda x} = \lambda_{\theta} e^{-\lambda_{\theta} x},$$

where $\lambda_{\theta} = \lambda - \theta$, and $-\infty < \theta < \lambda$.

Similarly, for the Gamma density

$$f(x) = \frac{\lambda^{\alpha} x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)},$$

we first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \mathbb{E}[e^{\theta X}] = \log \int_0^{\infty} e^{\theta x} \frac{\lambda^{\alpha} x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} dx = \alpha \log \left(\frac{\lambda}{\lambda - \theta} \right).$$

Then applying the tilting measure, we have

$$f_{\theta}(x) = e^{\theta x - \alpha \log\left(\frac{\lambda}{\lambda - \theta}\right)} \frac{\lambda^{\alpha} x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} = \left(\frac{\lambda - \theta}{\lambda} \right)^{\alpha} \frac{e^{\theta x} \lambda^{\alpha} x^{\alpha-1} e^{-\lambda x}}{\Gamma(\alpha)} = \frac{\lambda_{\theta}^{\alpha} x^{\alpha-1} e^{-\lambda_{\theta} x}}{\Gamma(\alpha)},$$

where again $\lambda_{\theta} = \lambda - \theta$.

(ii) We first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \sum_{k=0}^N e^{\theta k} \binom{N}{k} \alpha^k (1 - \alpha)^{N-k} = N \log [\alpha e^{\theta} + (1 - \alpha)] = N \log (1 - \alpha + \alpha e^{\theta}).$$

Then

$$\begin{aligned} p_{\theta}(k) &= e^{\theta k - N \log(1 - \alpha + \alpha e^{\theta})} \binom{N}{k} \alpha^k (1 - \alpha)^{N-k} = \frac{e^{\theta k} \binom{N}{k} \alpha^k (1 - \alpha)^{N-k}}{(1 - \alpha + \alpha e^{\theta})^N} \\ &= \binom{N}{k} \frac{(1 - \alpha)^{N-k}}{(1 - \alpha + \alpha e^{\theta})^{N-k}} \frac{(\alpha e^{\theta})^k}{(1 - \alpha + \alpha e^{\theta})^k} = \binom{N}{k} \alpha_{\theta}^k (1 - \alpha_{\theta})^{N-k}, \end{aligned}$$

where

$$\alpha_{\theta} = \frac{\alpha e^{\theta}}{1 - \alpha + \alpha e^{\theta}}, \quad 1 - \alpha_{\theta} = \frac{1 - \alpha}{1 - \alpha + \alpha e^{\theta}},$$

$\theta \in \mathbb{R}$. Therefore, we have verified p_{θ} is binomial (N, α_{θ}) .

(iii) We first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \sum_{k=0}^{\infty} e^{\theta k} \frac{e^{-\mu} \mu^k}{k!} = \log \left[e^{-\mu} \sum_{k=0}^{\infty} \frac{(\mu e^{\theta})^k}{k!} \right] = \log (e^{-\mu} e^{\mu e^{\theta}}) = \mu (e^{\theta} - 1).$$

Then

$$p_{\theta}(k) = e^{\theta k - \mu(e^{\theta} - 1)} \frac{e^{-\mu} \mu^k}{k!} = \frac{e^{\theta k} e^{-\mu e^{\theta}} \mu^k}{k!} = \frac{e^{-\mu e^{\theta}} (\mu e^{\theta})^k}{k!} = \frac{e^{-\mu_{\theta}} \mu_{\theta}^k}{k!},$$

where $\mu_{\theta} = \mu e^{\theta}$, and $\theta \in \mathbb{R}$. Therefore, we have verified p_{θ} is Poisson.

(iv) We first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \int_{-\infty}^{\infty} e^{\theta x} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} dx = \log \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi\sigma^2}} e^{\theta x - \frac{(x-\mu)^2}{2\sigma^2}} dx = \log e^{\mu\theta + \frac{\sigma^2\theta^2}{2}} = \mu\theta + \frac{\sigma^2\theta^2}{2}.$$

Then

$$\begin{aligned} f_{\theta}(x) &= e^{\theta x - \left(\mu\theta + \frac{\sigma^2\theta^2}{2}\right)} \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}} = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\theta x - \mu\theta - \frac{\sigma^2\theta^2}{2} - \frac{(x-\mu)^2}{2\sigma^2}} = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2 - 2x\mu + \mu^2 - 2\sigma^2\theta x + 2\mu\sigma^2\theta + \sigma^4\theta^2}{2\sigma^2}} \\ &= \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{x^2 - 2x(\mu + \sigma^2\theta) + (\mu + \sigma^2\theta)^2}{2\sigma^2}} = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{[x - (\mu + \sigma^2\theta)]^2}{2\sigma^2}}, \end{aligned}$$

i.e., f_{θ} is the density of $\mathcal{N}(\mu + \sigma^2\theta, \sigma^2)$ is verified.

(v) We first calculate $\kappa(\theta)$:

$$\kappa(\theta) = \log \int e^{\theta^T \mathbf{x}} \frac{1}{\sqrt{(2\pi)^k \det(\mathbf{C})}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T \mathbf{C}^{-1}(\mathbf{x}-\mu)} d\mathbf{x} = \log \left(e^{\theta^T \mu + \frac{\theta^T \mathbf{C} \theta}{2}} \right) = \theta^T \mu + \frac{\theta^T \mathbf{C} \theta}{2}.$$

Then

$$\begin{aligned} f_{\theta}(\mathbf{x}) &= e^{\theta^T \mathbf{x} - \left(\theta^T \mu + \frac{\theta^T \mathbf{C} \theta}{2} \right)} \frac{1}{\sqrt{(2\pi)^k \det(\mathbf{C})}} e^{-\frac{1}{2}(\mathbf{x}-\mu)^T \mathbf{C}^{-1}(\mathbf{x}-\mu)} \\ &= \frac{1}{\sqrt{(2\pi)^k \det(\mathbf{C})}} e^{\theta^T \mathbf{x} - \theta^T \mu - \frac{\theta^T \mathbf{C} \theta}{2} - \frac{1}{2}(\mathbf{x}-\mu)^T \mathbf{C}^{-1}(\mathbf{x}-\mu)} = \frac{1}{\sqrt{(2\pi)^k \det(\mathbf{C})}} e^{-\frac{1}{2}[\mathbf{x} - (\mu + \mathbf{C}\theta)]^T \mathbf{C}^{-1}[\mathbf{x} - (\mu + \mathbf{C}\theta)]} \end{aligned}$$

by similar process as the one-dimensional case, therefore, f_{θ} is the density of $\mathcal{N}(\mu + \mathbf{C}\theta, \mathbf{C})$ is verified.

□

EXERCISE 6.3. Let U be uniform(0, 1). Conditional on $U = u$, the number N of casualty insurance claims next week is Poisson with mean $15/(0.5 + u)$. Write $p = \mathbb{P}(N \geq 20)$.

1. Explain how to obtain a crude Monte Carlo estimator of p .
2. Develop an efficient simulator combining conditional expectation with a control variate.
3. Develop one combining conditional expectation with antithetic variables.
4. Write a program to determine the variances of all three estimators and compare their performance.

SOLUTION. (i) Draw U uniformly and generate a Poisson count with mean $\lambda(U) = 15/(0.5 + U)$ using the product-of-uniforms algorithm. Average the indicators $\mathbb{1}_{\{N \geq 20\}}$. Its variance is $p(1 - p)/n$.

(ii) Set $q(u) = \mathbb{P}(N \geq 20 \mid U = u)$. Compute this Poisson tail deterministically and use U as a control with known mean $1/2$. Average $q(U) + \alpha(U - 1/2)$. Estimate α from an independent pilot sample to preserve unbiasedness.

(iii) Average independent paired values $[q(U) + q(1 - U)]/2$. The pair is averaged; taking the logical union of two events estimates a different probability.

(iv) Estimate each variance from its independent replication values divided by the number of replications. Each antithetic replication uses two conditional evaluations, which should be counted when comparing computational cost.

```
import numpy as np
from scipy.stats import poisson
np.random.seed(12345)

def poisson_sample(lam):
    threshold = np.exp(-lam)
    product, k = 1.0, 0
    while product > threshold:
        k += 1
        product *= np.random.uniform()
    return k - 1
```

```

def crude(n):
    u = np.random.uniform(size=n)
    z = np.array([poisson_sample(15/(0.5+x)) >= 20 for x in u],
        dtype=float)
    return np.mean(z), np.var(z, ddof=1)/n

def conditional(u):
    return poisson.sf(19, 15/(0.5+u))

def conditional_control(n):
    pilot = np.random.uniform(size=10000)
    alpha = -np.cov(conditional(pilot), pilot, ddof=1)[0, 1] /
        np.var(pilot, ddof=1)
    u = np.random.uniform(size=n)
    z = conditional(u) + alpha*(u-0.5)
    return np.mean(z), np.var(z, ddof=1)/n

def conditional_antithetic(n):
    # n independent pairs, hence 2n conditional evaluations.
    u = np.random.uniform(size=n)
    z = (conditional(u) + conditional(1-u))/2
    return np.mean(z), np.var(z, ddof=1)/n

for name, method in [("Crude", crude), ("Conditional + control",
    conditional_control), ("Conditional + antithetic",
    conditional_antithetic)]:
    estimate, variance = method(10000)
    print(f"{name}: p = {estimate:.6f}, estimated variance =
        {variance:.8g}")

```

```

Crude: p = 0.290200, estimated variance = 2.0600456e-05
Conditional + control: p = 0.289498, estimated variance = 1.6078348e-06
Conditional + antithetic: p = 0.290073, estimated variance = 1.6000936e-06

```

□

EXERCISE 6.4. Let $Z_1, \dots, Z_n$ be independent standard normals and let h be increasing in each coordinate. Put $W = \sum_{i=1}^n a_i Z_i$ with all $a_i \geq 0$. Explain how to estimate $\mathbb{E}[h(Z_1, \dots, Z_n)]$ using stratified sampling on W , sampling W by inversion.

You may use the following fact: if Z is standard normal and independent of $X \sim N(\mu, \sigma^2)$, then $Z \mid X + Z = t$ is normal with mean $(t - \mu)/(1 + \sigma^2)$ and variance $\sigma^2/(1 + \sigma^2)$.

SOLUTION. Let $q = \sum_i a_i^2 > 0$. Choose equal-probability strata using $W/\sqrt{q}$, a standard normal. In stratum s , draw U_s uniformly from $((s-1)/S, s/S)$ and set $W_s = \sqrt{q} \Phi^{-1}(U_s)$. Independently generate $G \sim N(0, I)$ and set

$$Z = G + \frac{a}{q} (W_s - a^T G).$$

Then $a^T Z = W_s$ and Z has the required conditional normal distribution given $W = W_s$. Average $h(Z)$ in each stratum and weight by $p_s = 1/S$. Rejected draws must not be counted in a conditional average.

If every $a_i = 0$, W is constant and cannot provide stratification. The example uses the coordinatewise increasing function $h(z) = e^{0.2 \sum_i z_i}$.

```
import numpy as np
from scipy.stats import norm
np.random.seed(12345)

def h(z):
    return np.exp(0.2*np.sum(z))

def standard_normal(n):
    u = np.random.uniform(size=(n, 2))
    return np.sqrt(-2*np.log(u[:, 0])) * np.cos(2*np.pi*u[:, 1])

def stratified_sampling(n, S, R, a):
    a = np.asarray(a)
    q = np.dot(a, a)
    if q <= 0 or R < S:
        raise ValueError("Require nonzero a and at least one sample per
        ↪ stratum")
    counts = np.full(S, R//S)
    counts[:R % S] += 1
    estimate = 0.0
    for s in range(S):
        values = []
        for _ in range(counts[s]):
            u = (s + np.random.uniform())/S
            w = np.sqrt(q)*norm.ppf(u)
            g = standard_normal(n)
            z = g + a*(w-np.dot(a, g))/q
            values.append(h(z))
        estimate += np.mean(values)/S
    return estimate

n, S, R = 5, 10, 10000
a = np.arange(1, n+1, dtype=float)
estimate = stratified_sampling(n, S, R, a)
print(f"Estimated E[h(Z)]: {estimate:.6f}")
print(f"Exact E[h(Z)]: {np.exp(0.5*0.2**2*n):.6f}")
```

```
Estimated E[h(Z)]: 1.106091
Exact E[h(Z)]: 1.105171
```

□

EXERCISE 6.5. Suggest at least three variance-reduction methods for the Monte Carlo evaluation of

$$\int_0^\infty (x + 0.02x^2) \exp\{0.1\sqrt{1+\cos x} - x\} dx.$$

Interpret this integral as $\mathbb{E}[h(X)]$ for a suitable h and X . Include a method using control variates and one using stratification; choose a third method freely. Explain the methods and why variance reduction is expected.

SOLUTION. Write the integral as $z = \mathbb{E}[h(X)]$, where $X \sim \text{Exp}(1)$ and

$$h(x) = (x + 0.02x^2)e^{0.1\sqrt{1+\cos x}}.$$

- (i) *Control variates.* Use $W = X + 0.02X^2$, whose expectation is 1.04. It is close to $h(X)$, so average $h(X) + \alpha(W - 1.04)$ with a suitable fixed coefficient or one estimated from an independent pilot sample.
- (ii) *Stratification.* Partition the exponential probability scale into S equal intervals. In stratum s , draw U uniformly from $((s-1)/S, s/S)$ and set $X = -\log(1-U)$. Average $h(X)$ within each stratum and combine with weights $1/S$. This includes the unbounded final stratum without trying to sample uniformly on an infinite interval.
- (iii) *Importance sampling.* Use the mixture density

$$g(x) = \frac{xe^{-x} + 0.02x^2e^{-x}}{1.04}, \quad x > 0.$$

It mixes $\text{Gamma}(2, 1)$ and $\text{Gamma}(3, 1)$ with probabilities $1/1.04$ and $0.04/1.04$. Generate these integer-shape gammas as sums of independent exponential variables. The weighted integrand becomes

$$\frac{(x + 0.02x^2)e^{0.1\sqrt{1+\cos x} - x}}{g(x)} = 1.04e^{0.1\sqrt{1+\cos x}},$$

which lies between 1.04 and $1.04e^{0.1\sqrt{2}}$ and hence has small variance.

□

Derivative Estimation

Let's say that $Z = h(X)$. For *finite differences (FD)* and *infinitesimal perturbation analysis (IPA)*, we only assume the function h to depend on θ (structural dependence). For *likelihood ratio method (LR)*, we only assume the distribution of X to depend on θ (distributional dependence).

We want to find a random variable $D(\theta)$ such that

$$\mathbb{E}[D(\theta)] = z'(\theta) = \frac{d}{d\theta} \mathbb{E}[h_\theta(X)]$$

in case dependence is structural, and

$$\mathbb{E}[D(\theta)] = z'(\theta) = \frac{d}{d\theta} \mathbb{E}_\theta[h(X)]$$

in case dependence is distributional.

Then, if we have found an unbiased estimator, we could use crude Monte Carlo (or other methods) to estimate $z'(\theta)$: generate R copies $D_1(\theta), \dots, D_R(\theta)$, and compute $\frac{1}{R} \sum_{i=1}^R D_i(\theta)$.

SECTION 7.1

| Finite Differences

Suppose that for each θ , we can generate a random variable $Z(\theta)$ with expectation $z(\theta)$. Starting point is the definition of a derivative:

$$f'(\theta) = \lim_{h \rightarrow 0} \frac{f(\theta + h) - f(\theta)}{h} = \lim_{h \rightarrow 0} \frac{f\left(\theta + \frac{h}{2}\right) - f\left(\theta - \frac{h}{2}\right)}{h}.$$

This suggests two possible derivative estimators, namely based on

$$\tilde{D}(\theta) = \frac{Z(\theta + h) - Z(\theta)}{h}, \quad D(\theta) = \frac{Z\left(\theta + \frac{h}{2}\right) - Z\left(\theta - \frac{h}{2}\right)}{h}.$$

To check the bias, apply Taylor series about θ :

$$\begin{aligned} \mathbb{E}[\tilde{D}(\theta)] &= \frac{z(\theta + h) - z(\theta)}{h} = \frac{1}{h} \left(z'(\theta)h + z''(\theta)\frac{h^2}{2} + z'''(\theta)\frac{h^3}{6} + \dots \right), \\ \mathbb{E}[D(\theta)] &= \frac{z\left(\theta + \frac{h}{2}\right) - z\left(\theta - \frac{h}{2}\right)}{h} = \frac{1}{h} \left(z'(\theta)h + z'''(\theta)\frac{h^3}{24} + \dots \right). \end{aligned}$$

The bias of $D(\theta)$ is an order of magnitude lower.

Remark. (i) On one hand, don't pick h too large, because it induces bias. The bias will vanish as $h \rightarrow 0$. On other hand, don't pick h too small, because it increases variance.
(ii) One can use common random numbers to reduce variance.

EXAMPLE 7.1. Suppose that $Z = h(X)$, where X is $\exp(\theta)$ distributed and $h(x) = x^p$ with $p \in \mathbb{N}$. Note that $z(\theta) = \mathbb{E}[X^p]$ is the p -th moment of the $\exp(\theta)$ distribution, i.e.,

$$z(\theta) = p! \left(\frac{1}{\theta} \right)^p,$$

but let's suppose we unknowingly wish to estimate $z'(\theta)$. Recall that the quantile function of X is given by

$$F^{\leftarrow}(x) = -\frac{\log(1-x)}{\theta}.$$

This leads to

$$\begin{aligned} D(\theta) &= \frac{Z\left(\theta + \frac{h}{2}\right) - Z\left(\theta - \frac{h}{2}\right)}{h} \\ &= \frac{1}{h} \left(\left(-\frac{\log(1-U)}{\theta + \frac{h}{2}} \right)^p - \left(-\frac{\log(1-U)}{\theta - \frac{h}{2}} \right)^p \right) = \frac{1}{h} \left(\left(\frac{1}{\theta + \frac{h}{2}} \right)^p - \left(\frac{1}{\theta - \frac{h}{2}} \right)^p \right) (-\log(1-U))^p, \end{aligned}$$

where U is standard uniformly distributed. With many samples of U , many samples of $D(\theta)$ can be generated, after which the sample mean gives a biased estimation of $z'(\theta)$.

Common random variables versus independent sampling can be seen that using this estimator,

$$\text{Var}(D(\theta)) = \frac{1}{h^2} \left(\left(\frac{1}{\theta + \frac{h}{2}} \right)^p - \left(\frac{1}{\theta - \frac{h}{2}} \right)^p \right)^2 ((2p)! - (p!)^2).$$

However, in case we wouldn't have used common random numbers but sampled independently, we would have faced

$$\text{Var}(D(\theta)) = \frac{1}{h^2} \left(\left(\frac{1}{\theta + \frac{h}{2}} \right)^{2p} + \left(\frac{1}{\theta - \frac{h}{2}} \right)^{2p} \right) ((2p)! - (p!)^2),$$

which is larger.

SECTION 7.2

| Infinitesimal Perturbation Analysis

Suppose that $Z = h_\theta(X)$ (structural dependence), and that we want to know $\frac{d}{d\theta}\mathbb{E}[Z]$. Let $D(\theta) := \frac{d}{d\theta}h_\theta(X)$, then *infinitesimal perturbation analysis (IPA)* is based on the following assumption:

$$\frac{d}{d\theta}\mathbb{E}[h_\theta(X)] = \mathbb{E}\left[\frac{d}{d\theta}h_\theta(X)\right] = \mathbb{E}[D(\theta)].$$

The IPA method simply implies the crude Monte Carlo simulation of $D(\theta)$: sample R copies of $D(\theta)$, and create confidence intervals based on these copies using regular methods and techniques.

EXAMPLE 7.2. Suppose that $Z = g(X)$, where X is $\exp(\theta)$ distributed and $g(x) = x^p$ with $p \in \mathbb{N}$. Z has same distribution as $h_\theta(U)$, where

$$h_\theta(x) = \left(-\frac{\log(x)}{\theta}\right)^p,$$

and U is standard uniform. We have

$$D(\theta) = \frac{d}{d\theta}h_\theta(x) = p\left(-\frac{\log(x)}{\theta}\right)^{p-1}\frac{\log(x)}{\theta^2} = -\frac{ph_\theta(x)}{\theta}.$$

Thus we create replicates of $h_\theta(U)$ using known methods, and then multiply each with $-\frac{p}{\theta}$. These are the resulting replicates for $D(\theta)$. Taking a sample mean provides an unbiased estimation for $z'(\theta)$.

PROPOSITION 7.3.

EXAMPLE 7.4.

SECTION 7.3

Likelihood Ratio Method

For this method to work, we assume the dependence on θ to be distributional, i.e.,

$$Z(\theta) = h(X), \quad z(\theta) = \mathbb{E}_\theta[h(X)]$$

where X has a density $f_\theta(x)$, and we wish to estimate $z'(\theta)$ in the point θ_0 . The likelihood ratio method is reminiscent of importance sampling. Let $L(\theta, x) = \frac{f_\theta(x)}{f_{\theta_0}(x)}$, then

$$z(\theta) = \int h(x)f_\theta(x) dx = \int h(x)L(\theta, x)f_{\theta_0}(x) dx = \mathbb{E}_{\theta_0}[h(X)L(\theta, X)],$$

where $\mathbb{E}_{\theta_0}[\cdot]$ indicates expectation with respect to the density f_{θ_0} . This suggests that

$$z'(\theta) = \mathbb{E}_{\theta_0}[h(X)L'(\theta, X)],$$

where $L'(\theta, X) = \frac{d}{d\theta}L(\theta, X)$. We can write

$$z'(\theta_0) = \mathbb{E}_{\theta_0}[h(X)S_{\theta_0}(X)],$$

where

$$S_{\theta_0}(x) = \left. \frac{d}{d\theta}L(\theta, x) \right|_{\theta=\theta_0} = \left. \frac{f'_\theta(x)}{f_\theta(x)} \right|_{\theta=\theta_0}.$$

We will refer to $S_{\theta_0}(x)$ as the *score function* evaluated at $\theta = \theta_0$. And all this suggests the following unbiased strategy for estimating $z'(\theta_0)$:

$$D(\theta_0) = h(X)S_{\theta_0}(X).$$

Hence, we generate R *i.i.d.* samples of X , where X has density f_{θ_0} , then compute $\frac{1}{R} \sum_{i=1}^R h(X_i)S_{\theta_0}(X_i)$. Confidence intervals can be computed by the known techniques.

For a single random variable X ,

$$S_{\theta_0}(x) = \left. \frac{d}{d\theta} \log f_\theta(x) \right|_{\theta=\theta_0}.$$

EXAMPLE 7.5. Suppose that $Z = h(X)$, where X is $\exp(\theta)$ distributed and $h(x) = x^p$ with $p \in \mathbb{N}$. The corresponding score function is

$$S_{\theta_0}(x) = \left. \frac{d}{d\theta} \log f_\theta(x) \right|_{\theta=\theta_0} = \frac{1}{\theta_0} - x.$$

Hence,

$$h(x)S_{\theta_0}(x) = \frac{x^p}{\theta_0} - x^{p+1}.$$

So, to obtain an unbiased estimation of the derivative of z at $\theta = \theta_0$, we generate $X_1, \dots, X_R$ from the $\exp(\theta_0)$ distribution, and compute

$$\frac{1}{R} \sum_{i=1}^R \left(\frac{X_i^p}{\theta_0} - X_i^{p+1} \right).$$

EXAMPLE 7.6.

PROPOSITION 7.7.

Remark. Each of these methods has its drawbacks and advantages.

- (i) FD is easiest to grasp, and virtually always works, but yields biased estimators. IPA and LR are unbiased.*
- (ii) IPA often yields the smallest variance, but cannot always be applied.*
- (iii) LR has a broader scope than IPA.*

Exercises

EXERCISE 7.1. Let X have Weibull density $f_X(x) = 2\lambda^2 x e^{-\lambda^2 x^2} \mathbb{1}_{\{x>0\}}$, and let $Z = h(X)$ with $h(x) = 2^x$. Estimate $\frac{d}{d\lambda} \mathbb{E}[Z] \big|_{\lambda=1/2}$.

- (i) Explain finite differences without common random numbers.
- (ii) Explain finite differences with common random numbers.
- (iii) Explain infinitesimal perturbation analysis.
- (iv) Explain the likelihood ratio method.
- (v) Implement each method and compare estimator quality, for example through sample mean squared errors. Rank the methods and justify your findings.

SOLUTION. (i) To estimate $\frac{d}{d\lambda} \mathbb{E}[Z] \big|_{\lambda=\frac{1}{2}}$ using the *method of finite differences (FD)* without common random numbers, we can apply the formula:

$$z'(\lambda) \approx \frac{\mathbb{E}\left[Z\left(\lambda + \frac{h}{2}\right)\right] - \mathbb{E}\left[Z\left(\lambda - \frac{h}{2}\right)\right]}{h},$$

where h is a small step size, and $Z(\lambda) = h(X_\lambda)$ with X following a Weibull distribution with parameters 2 and λ . Notice that h should be small enough to ensure a good approximation of the derivative while balancing the trade-off between variance and bias. The *mean squared error (MSE)* of the estimator is optimised when choosing

$$h = \frac{1}{R^{\frac{1}{6}}} \frac{(576 \text{Var}[Z(\lambda)])^{\frac{1}{6}}}{|z'''(\lambda)|^{\frac{1}{3}}}.$$

In the comparison below, we take $h = 0.01$ and evaluate the MSE against a numerical-quadrature benchmark. Then we can calculate each $z(\lambda) = \mathbb{E}[Z(\lambda)]$ and estimate $\frac{d}{d\lambda} \mathbb{E}[Z] \big|_{\lambda=\frac{1}{2}}$ by

$$\frac{z\left(\frac{1+h}{2}\right) - z\left(\frac{1-h}{2}\right)}{h}.$$

The following is the code that implements the above process.

```
import numpy as np
np.random.seed(12345)
from scipy.integrate import quad
def true_derivative(lam):
    b = np.log(2)
    integral = quad(lambda t: t*t*np.exp(b*t/lam-t*t), 0, np.inf)[0]
    return -2*b*integral/lam**2

def inverse_transform(lam, u):
    k = 2
    return (-np.log(1 - u))**(1/k) / lam

def weibull(lam, size=1):
    uniform_samples = np.random.uniform(0, 1, size)
    return inverse_transform(lam, uniform_samples)

def h(x):
    return 2**x
```

```

def simulate(lam, h, number_samples=10000):
    weibull_samples = weibull(lam, number_samples)
    z_values = h(weibull_samples)
    return np.mean(z_values)

def finite_difference(lam, h, step_size):
    return (simulate(lam + step_size/2, h) - simulate(lam - step_size/2,
    ↪ h)) / step_size

lam = 0.5
size = 0.01
num = 1000

derivatives = []
for _ in range(num):
    derivative_estimate = finite_difference(lam, h, size)
    derivatives.append(derivative_estimate)

mse = np.mean((np.asarray(derivatives) - true_derivative(lam))**2)

print(f"Estimated derivative: {np.mean(derivatives)}")
print(f"Sample Mean Squared Error: {mse}")

```

An output of this code is as follows.

```

Estimated derivative: -14.73304029689869
Sample Mean Squared Error: 30.458229192339576

```

- (ii) We follow a similar procedure as (i) but now use common random numbers to reduce variance. The following is the code that implements the above process.

```

import numpy as np
np.random.seed(12345)
from scipy.integrate import quad
def true_derivative(lam):
    b = np.log(2)
    integral = quad(lambda t: t*t*np.exp(b*t/lam-t*t), 0, np.inf)[0]
    return -2*b*integral/lam**2

def inverse_transform(lam, u):
    k = 2
    return (-np.log(1 - u))**(1/k) / lam

def common_uniform(size=1):
    return np.random.uniform(0, 1, size)

def h(x):
    return 2**x

def simulate(lam, h, uniform_samples):

```

```

weibull_samples = inverse_transform(lam, uniform_samples)
z_values = h(weibull_samples)
return np.mean(z_values)

def finite_difference(lam, h, step_size, num_samples=10000):
    uniform_samples = common_uniform(num_samples)
    return (simulate(lam + step_size/2, h, uniform_samples) - simulate(lam
↪ - step_size/2, h, uniform_samples)) / step_size

lam = 0.5
size = 0.01
num = 1000

derivatives = []
for _ in range(num):
    derivative_estimate = finite_difference(lam, h, size)
    derivatives.append(derivative_estimate)

mse = np.mean((np.asarray(derivatives) - true_derivative(lam))**2)

print(f"Estimated derivative: {np.mean(derivatives)}")
print(f"Sample Mean Squared Error: {mse}")

```

An output of this code is as follows.

```

Estimated derivative: -14.972753725974322
Sample Mean Squared Error: 0.06791156846184157

```

(iii) *Infinitesimal perturbation analysis (IPA)* involves analysing how small change of λ affect Z . We have

$$\frac{d}{d\lambda} \mathbb{E}[h_\lambda(X)] = \mathbb{E} \left[\frac{d}{d\lambda} h_\lambda(X) \right] = \mathbb{E}[D(\lambda)],$$

where $D(\lambda) = \frac{d}{d\lambda} h_\lambda(X)$. The inverse representation is $X_\lambda = \sqrt{-\log(1-U)}/\lambda$, so

$$D(\lambda) = -\frac{\log 2}{\lambda} X_\lambda 2^{X_\lambda}.$$

The code evaluates this pathwise derivative directly.

Remark. The interchange of expectation and derivative is allowed if $Z(\lambda)$ is a.s. differentiable at λ_0 and that a.s. $Z(\lambda)$ satisfies the Lipschitz condition $|Z(\lambda_1) - Z(\lambda_2)| \leq |\lambda_1 - \lambda_2| M$ for λ_1, λ_2 in a non-random neighbourhood of λ_0 , where $\mathbb{E}[M] < \infty$.

```

import numpy as np
np.random.seed(12345)
from scipy.integrate import quad
def true_derivative(lam):
    b = np.log(2)
    integral = quad(lambda t: t*t*np.exp(b*t/lam-t*t), 0, np.inf)[0]
    return -2*b*integral/lam**2

```

```

def inverse_transform(lam, u):
    k = 2
    return (-np.log(1 - u))**(1/k) / lam

def h(x):
    return 2**x

def derivative(lam, u):
    x = inverse_transform(lam, u)
    return -np.log(2)*x*2**x/lam

def ipa(lam, num_samples=10000):
    uniform_samples = np.random.uniform(0, 1, num_samples)
    derivatives = derivative(lam, uniform_samples)
    return np.mean(derivatives)

lam = 0.5
num = 1000

estimates = []
for _ in range(num):
    estimate = ipa(lam)
    estimates.append(estimate)

mse = np.mean((np.asarray(estimates) - true_derivative(lam))**2)

print(f"Estimated derivative: {np.mean(estimates)}")
print(f"Sample Mean Squared Error: {mse}")

```

An output of this code is as follows.

```

Estimated derivative: -14.966937662766718
Sample Mean Squared Error: 0.06765609389381232

```

- (iv) The *likelihood ratio method (LR)* involves expressing the derivative of the expectation as an expectation itself. We have

$$z(\lambda) = \int h(x)L(\lambda, x)f_{\lambda_0}(x) dx = \mathbb{E}_{\lambda_0}[h(x)L(\lambda, x)],$$

where the likelihood ratio

$$L(\lambda, x) = \frac{f_{\lambda}(x)}{f_{\lambda_0}(x)},$$

$\mathbb{E}_{\lambda_0}$ indicates expectation with respect to the density f_{λ_0} . Then we can write

$$z'(\lambda_0) = \mathbb{E}_{\lambda_0}[h(X)S_{\lambda_0}(X)],$$

where the score function

$$S_{\lambda_0}(x) = \frac{2}{\lambda_0} - 2\lambda_0 x^2.$$

Then we generate R i.i.d. samples of X from the Weibull distribution with $\lambda_0 = \frac{1}{2}$, and for each sample X_i , calculate $S_{\lambda_0}(X_i)$. Then the derivative is estimated by

$$\widehat{z}'(\lambda_0) = \frac{1}{R} \sum_{i=1}^R h(X_i) S_{\lambda_0}(X_i).$$

The following is the code that implements the above process.

```
import numpy as np
np.random.seed(12345)
from scipy.integrate import quad
def true_derivative(lam):
    b = np.log(2)
    integral = quad(lambda t: t*t*np.exp(b*t/lam-t*t), 0, np.inf)[0]
    return -2*b*integral/lam**2

def inverse_transform(lam, u):
    k = 2
    return (-np.log(1 - u))**(1/k) / lam

def h(x):
    return 2**x

def score_function(x, lam, lam0):
    k = 2
    return 2/lam0 - 2*lam0*x**2

def likelihood_ratio_method(lam0, lam, num_samples=10000):
    uniform_samples = np.random.uniform(0, 1, num_samples)
    x_values = inverse_transform(lam0, uniform_samples)
    score_values = score_function(x_values, lam, lam0)
    z_values = h(x_values)
    return np.mean(z_values * score_values)

lam_0 = 0.5
lam = 0.5
num_simulations = 1000

derivatives = []
for _ in range(num_simulations):
    derivative_estimate = likelihood_ratio_method(lam_0, lam)
    derivatives.append(derivative_estimate)

mse = np.mean((np.asarray(derivatives) - true_derivative(lam))**2)

print(f"Estimated derivative: {np.mean(derivatives)}")
print(f"Sample Mean Squared Error: {mse}")
```

An output of this code is as follows.

```
Estimated derivative: -14.976831624167014  
Sample Mean Squared Error: 0.5950572279073981
```

- (v) The MSE is measured against the derivative obtained by numerical quadrature, rather than against the sample mean of the estimates. All methods use the same sample size and number of independent replications. Common random numbers sharply reduce the FD variance, and the analytic IPA avoids FD bias. In this run, IPA and FD with common random numbers have very similar MSE, followed by LR and then FD with independent samples. The derivative is negative because increasing λ decreases X_λ .

□

Simulation-Based Optimisation

Consider the problem:

$$\min_{\theta \in \Theta} z(\theta) = \min_{\theta \in \Theta} \mathbb{E}[Z(\theta)]$$

with $\Theta \subseteq \mathbb{R}$. Dependence is either structural or distributional. We assume that $z(\cdot)$ is a smooth function, and that the constraint is not binding, so that $\nabla z(\theta^*) = 0$. We will focus on the steepest descent algorithm: recursively construct sequence $\theta_0, \theta_1, \theta_2, \dots$ of proposal solutions through

$$\theta_{n+1} = \Pi_{\Theta}(\theta_n - \epsilon_n z'(\theta_n)),$$

where ϵ_n determines step size at n -th iteration (in machine learning also called the learning rate). The function

$$\Pi_{\Theta}(x) = \arg \min_{y \in \Theta} |x - y|$$

takes care that $\theta_{n+1} \in \Theta$. Instead of the deterministic $z'(\theta_n)$, use the random variable $D(\theta_n)$. When using *FD* (biased estimates), such algorithms are called *Kiefer-Wolfowitz* algorithms. When using *IPA* and *LR* (unbiased estimates), such algorithms are called *Robbins-Monro* algorithms.

THEOREM 8.1 (Robbins-Monro Algorithms).

Let Θ be a compact interval and z a continuously differentiable, strictly convex function with a unique minimizer θ^ . Suppose D_n is conditionally unbiased for $z'(\theta_n)$ given the past, and its conditional second moment is uniformly bounded. Let the positive step sizes satisfy*

$$\sum_{n=0}^{\infty} \epsilon_n = \infty, \quad \sum_{n=0}^{\infty} \epsilon_n^2 < \infty.$$

Then the projected iteration

$$\theta_{n+1} = \Pi_{\Theta}(\theta_n - \epsilon_n D_n)$$

converges almost surely and in mean square to the minimizer:

$$\theta^* = \arg \min_{\theta \in \Theta} z(\theta), \quad \lim_{n \rightarrow \infty} \mathbb{E}[(\theta_n - \theta^*)^2] = 0.$$

Hence, under these assumptions, θ_n for large n should be a good approximation of the optimizer: continue until there is hardly improvement. A similar theorem for *Kiefer-Wolfowitz* algorithms exists, but additional conditions, e.g., on h , are required.

A natural choice used throughout the literature for the step sizes is $\epsilon_n = \frac{C}{n+1}$ for some $C > 0$.

EXAMPLE 8.2. Consider the problem: $\min_{\theta} \mathbb{E}[(X - \theta)^2]$ for $X \sim \mathcal{N}(\mu, \sigma^2)$. Obviously, the minimum is σ^2 at $\theta^* = \mu$. Suppose, though, we don't know that.

- (i) We can use stochastic optimisation together with *FD*. That is, iterate

$$\theta_{n+1} = \Pi_{\Theta}(\theta_n - \epsilon_n D(\theta_n))$$

for some θ_0 with, letting $X_1, X_2, \dots$ be $\mathcal{N}(\mu, \sigma^2)$ distributed samples,

$$D(\theta_n) = \frac{1}{R} \sum_{i=1}^R \frac{\left(X_i - \left(\theta + \frac{h}{2}\right)\right)^2 - \left(X_i - \left(\theta - \frac{h}{2}\right)\right)^2}{h}.$$

- (ii) We can also use stochastic optimisation together with *IPA*. That is, iterate

$$\theta_{n+1} = \Pi_{\Theta}(\theta_n - \epsilon_n D(\theta_n))$$

for some θ_0 with $X_1, X_2, \dots$ be $\mathcal{N}(\mu, \sigma^2)$ distributed samples,

$$D(\theta_n) = -\frac{1}{R} \sum_{i=1}^R 2(X_i - \theta).$$

- (iii) We can also use stochastic optimisation together with *LR*. We first need to make the structural dependence distributional:

$$\mathbb{E}[(X - \theta)^2] = \mathbb{E}_{\theta}[Y^2]$$

with $Y = X - \theta \sim \mathcal{N}(\mu - \theta, \sigma^2)$. Since

$$f_{\theta}(y) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(y-(\mu-\theta))^2}{2\sigma^2}},$$

we have

$$S_{\theta}(y) = \frac{d}{d\theta} \log f_{\theta}(y) = -\frac{y - (\mu - \theta)}{\sigma^2}.$$

Hence, with $Y_1, Y_2, \dots$ be $\mathcal{N}(\mu - \theta_n, \sigma^2)$ distributed samples,

$$D(\theta_n) = -\frac{1}{R} \sum_{i=1}^R \frac{(Y_i - (\mu - \theta_n))Y_i^2}{\sigma^2}.$$

So, in short, the idea is to mimic steepest descent/gradient descent, and just use derivative estimation methods for the gradient. While of intrinsic interest to the stochastic simulation community, these algorithms are also at the basis for optimisation algorithms in machine learning. For instance, this algorithm is widely used to train neural networks.

The following method is especially useful for sample path optimisation, and we make similar assumptions as before (smoothness and convexity; constraints not binding).

EXAMPLE 8.3 (Queuing Theory).

Exercises

EXERCISE 8.1. Consider

$$\min_{\theta \in [0,1]} \mathbb{P}(\max \{ \theta X_1 + X_2, (1 - \theta) X_3 \} \geq 3),$$

where $X_1 \sim \text{Gamma}(2, 2)$, $X_2 \sim \text{Gamma}(2, 1)$, and $X_3 \sim \text{Gamma}(2, 2/3)$. Assume the minimum is attained in the interior. Use the iteration

$$\theta_{n+1} = \Pi_{[0,1]} \left(\theta_n - \frac{C}{n+1} D(\theta_n) \right),$$

where D is a derivative estimate.

- (i) Use likelihood ratio derivative estimates with $R = 10$, $C = 0.2$, and $\theta_0 = 0.5$. Plot θ_n for $n = 0, \dots, 50000$ and report the estimated optimum.
- (ii) Experiment with other values of C . Discuss whether results improve and the tradeoff in choosing C .

SOLUTION. (i) Here $\text{Gamma}(k, s)$ uses shape k and rate s . For shape two, generate $X = -\log(U_1 U_2)/s$. To apply LR, put $Y_1 = \theta X_1$ and $Y_3 = (1 - \theta) X_3$, so that the event $\max \{Y_1 + X_2, Y_3\} \geq 3$ no longer depends structurally on θ . For $0 < \theta < 1$, its score is

$$S_\theta = \frac{2X_1 - 2}{\theta} + \frac{2 - (2/3)X_3}{1 - \theta}.$$

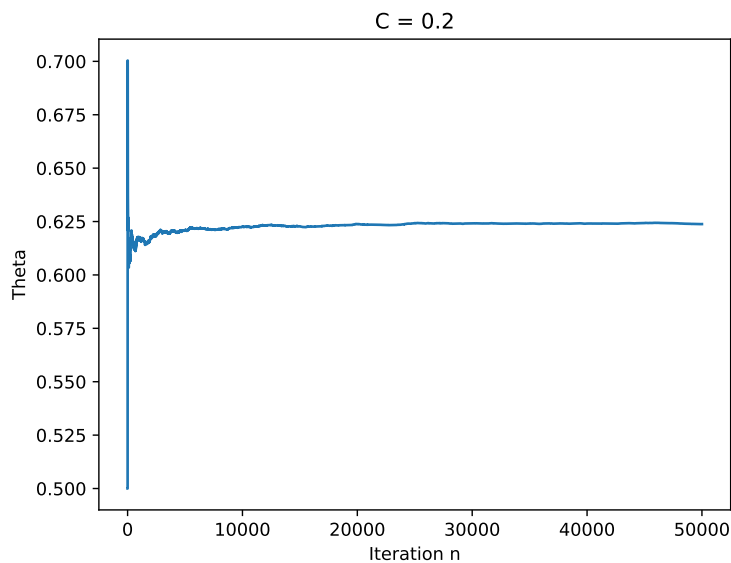
Average the event indicator times this score over $R = 10$ samples. Differentiating a gamma shape parameter would estimate the wrong derivative. Zero-mean score controls subtract the event with Y_1 or Y_3 omitted to reduce variance. A vanishing numerical guard keeps the iteration away from the singular endpoints of the score.

```
import numpy as np
import matplotlib.pyplot as plt
np.random.seed(12345)

def gamma_two(rate, size):
    u = np.random.uniform(size=(size, 2))
    return -np.log(u).sum(axis=1)/rate

def derivative(theta, R):
    x1, x2, x3 = gamma_two(2, R), gamma_two(1, R), gamma_two(2/3, R)
    y1, y3 = theta*x1, (1-theta)*x3
    event = np.maximum(y1+x2, y3) >= 3
    baseline1 = np.maximum(x2, y3) >= 3
    baseline3 = y1+x2 >= 3
    term1 = (event.astype(float)-baseline1)*(2*x1-2)/theta
    term3 = (event.astype(float)-baseline3)*(2-(2/3)*x3)/(1-theta)
    return np.mean(term1+term3)

def optimize(C, iterations=50000, R=10):
    theta = 0.5
    values = [theta]
    # Stay inside the domain of the LR score, with a vanishing numerical
    # ↪ guard.
    for n in range(iterations):
```



```

    update = theta - C/(n+1)*derivative(theta, R)
    guard = 1e-8/(n+1)
    theta = np.clip(update, guard, 1-guard)
    values.append(theta)
    return np.asarray(values)

constants = [0.2, 0.05, 0.1, 0.3, 0.5, 1.0, 2.0]
for j, C in enumerate(constants):
    np.random.seed(12345)
    values = optimize(C)
    plt.figure()
    plt.plot(values)
    plt.xlabel("Iteration n")
    plt.ylabel("Theta")
    plt.title(f"C = {C}")
    path = "HS4_Q2_1.pdf" if j == 0 else f"HS4_Q2_2_{j}.pdf"
    plt.savefig(path)
    plt.close()
    print(f"C = {C}: theta = {values[-1]:.6f}")

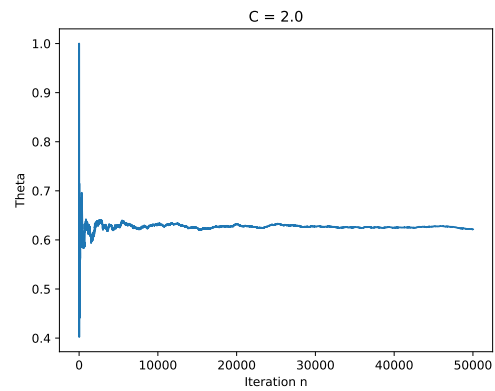
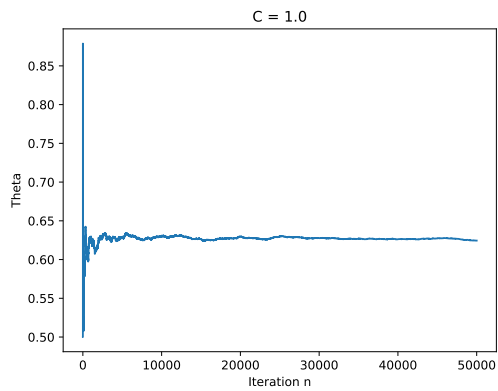
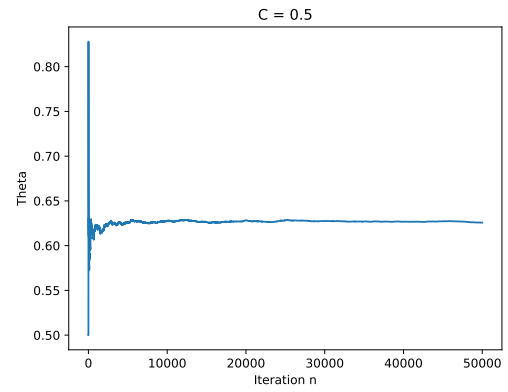
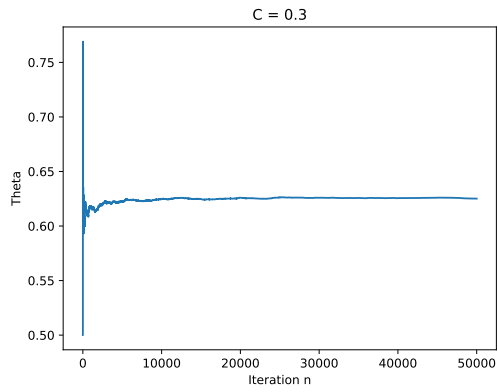
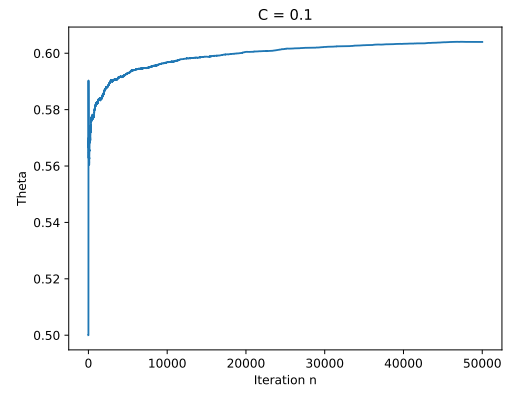
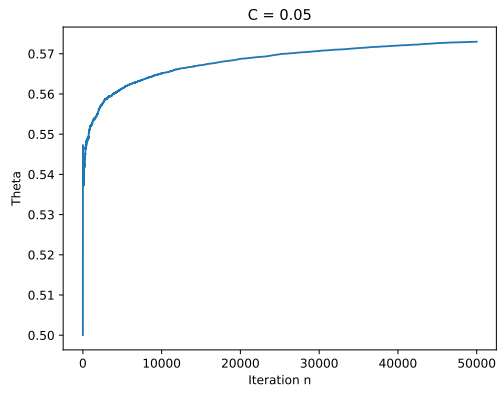
```

```

C = 0.2: theta = 0.623807
C = 0.05: theta = 0.572996
C = 0.1: theta = 0.604003
C = 0.3: theta = 0.625128
C = 0.5: theta = 0.625741
C = 1.0: theta = 0.624478
C = 2.0: theta = 0.621851

```

- (ii) Smaller C reduces early fluctuations but may converge slowly. Larger C moves faster and may push the iterate close to a boundary, where the LR variance is large. Compare several runs and the objective value, rather than interpreting a single final iterate as an exact optimum.



□

Rare-Event Simulation

In rare-event simulation, we consider the problem of estimating a rare event A . Think of $z := \mathbb{P}(A)$ as being in the order of 10^{-3} or less.

Naive thought: crude Monte Carlo simulation. Let $Z = \mathbb{1}_{\{A\}}$, so that indeed $z = \mathbb{E}[Z] = \mathbb{P}(A)$. Z is then Bernoulli distributed with parameter z . We know that $\text{Var}(Z) = z(1 - z)$. This means that

$$\frac{\sigma_Z}{z} = \sqrt{\frac{1-z}{z}}$$

which behaves like $z^{-\frac{1}{2}}$ for small z . Recall the 95%-confidence interval for z :

$$\left(\hat{z}_R - 1.96 \frac{\sigma_Z}{\sqrt{R}}, \hat{z}_R + 1.96 \frac{\sigma_Z}{\sqrt{R}} \right).$$

Suppose that we want to acquire a precision such that the width of our confidence interval is about 20% of the value of z . In other words:

$$0.1z = 1.96 \frac{\sigma_Z}{\sqrt{R}} \quad \Rightarrow \quad R = \frac{100 \cdot 1.96^2 z(1-z)}{z^2}.$$

This number increases like z^{-1} towards ∞ as $z \downarrow 0$. Thus there is always a point at which crude Monte Carlo will fail as R gets too large.

Formal Setup. Let $A(x)$ be a family of rare events, with e.g., $x \in \mathbb{R}^+$ or $x \in \mathbb{N}$.

$$z(x) := \mathbb{P}(A(x)) \rightarrow 0$$

as $x \rightarrow \infty$. Let $Z(x)$ be an unbiased estimator for $z(x)$ such that $\mathbb{E}[Z(x)] = z(x)$. An algorithm is defined as a family $Z(x)$ of such estimators.

We want to find well-performing algorithms such that the required R does not explode, i.e., we wish R to stay finite as $z(x) \downarrow 0$. This happens, when $Z(x)$ has a *bounded relative error*:

$$\limsup_{x \rightarrow \infty} \frac{\text{Var}(Z(x))}{z(x)^2} < \infty.$$

In practice, we often check whether a variant of this condition holds, *logarithmic efficiency*:

$$\limsup_{x \rightarrow \infty} \frac{\text{Var}(Z(x))}{z(x)^{2-\epsilon}} = 0$$

for any $\epsilon > 0$, which is a slightly weaker condition. Since $z(x) \rightarrow 0$ as $x \rightarrow \infty$, $\lim_{x \rightarrow \infty} z(x)^\epsilon = 0$, the former implies the latter.

EXAMPLE 9.1. Let N be a geometric random variable with success parameter π , i.e. $\mathbb{P}(N = n) = \pi(1 - \pi)^{n-1}$, and consider $z := \mathbb{P}(N \leq m) = 1 - (1 - \pi)^m$.

Now, let $\pi = \pi(x)$ depend on a parameter x such that $\pi(x) \downarrow 0$ as $x \rightarrow \infty$ so that $z = z(x) \downarrow 0$. For simulating z , we consider importance sampling such that N is instead simulated from a geometric distribution with a success parameter $\tilde{\pi}$, which does not depend on x . A family $\{Z(x)\}$ of estimators is now given by

$$Z(x) = \frac{1}{R} \sum_{i=1}^R \mathbb{1}_{\{N_i \leq m\}} \frac{\pi(x)(1 - \pi(x))^{N_i-1}}{\tilde{\pi}(1 - \tilde{\pi})^{N_i-1}},$$

where N_i are sampled from the fixed proposal and m is fixed. Put $q_n = \tilde{\pi}(1 - \tilde{\pi})^{n-1}$. Since $z(x) \sim m\pi(x)$,

$$\lim_{x \rightarrow \infty} \frac{\text{Var}(Z(x))}{z(x)^2} = \frac{1}{R} \left(\frac{1}{m^2} \sum_{n=1}^m \frac{1}{q_n} - 1 \right) < \infty.$$

Thus this fixed proposal has bounded relative error; the constant depends on m , R and $\tilde{\pi}$.

We now proceed to the study of sums of light-tailed random variables (i.e., the relevant tails decay at least at an exponential rate). Let $X_1, X_2, \dots$ be *i.i.d.* random variables with support $\mathbb{R}$ and $\mathbb{E}[e^{\theta X_1}] < \infty$ for some $\theta > 0$. Furthermore, let

$$S_n = \sum_{i=1}^n X_i, \quad \tau(x) := \inf \{n : S_n > x\}.$$

Consider the problem: find $\mathbb{P}(\tau(x) < \infty)$ for some $x > 0$.

Solution is easy when $\mathbb{E}[X_1] > 0$: S_n will grow bigger and eventually reach x : probability is one. When $\mathbb{E}[X_1] < 0$: S_n will grow smaller and may not reach x : probability may be smaller than one, so we use exponential tilting to change the density function of X_1 .

Recall that

$$L_{n,\theta} = \frac{f^n(\mathbf{X})}{g_\theta^n(\mathbf{X})} = \prod_{i=1}^n \frac{f(X_i)}{g_\theta(X_i)} = e^{-\theta S_n} \hat{F}(\theta)^n = e^{-\theta S_n + n\kappa(\theta)},$$

where $S_n = \sum_{i=1}^n X_i$, $\hat{F}(\theta) = \mathbb{E}[e^{\theta X_1}]$, and $\kappa(\theta) = \log \hat{F}(\theta)$. Moreover,

$$\mathbb{E}[h(X_1, \dots, X_n)] = \mathbb{E}_\theta[h(X_1, \dots, X_n) L_{n,\theta}] = \mathbb{E}_\theta[h(X_1, \dots, X_n) e^{-\theta S_n + n\kappa(\theta)}].$$

Now: *Siegmund's algorithm* to consider the problem of estimating, for large x ,

$$z(x) = \mathbb{P}(\tau(x) < \infty), \quad \tau(x) := \inf \{n : S_n > x\}.$$

We use exponential tilting to obtain

$$z(x) = \mathbb{E}[\mathbb{1}_{\{\tau(x) < \infty\}}] = \mathbb{E}_\theta[\mathbb{1}_{\{\tau(x) < \infty\}} L_{\tau(x),\theta}] = \mathbb{E}_\theta[\mathbb{1}_{\{\tau(x) < \infty\}} e^{-\theta S_{\tau(x)} + \tau(x)\kappa(\theta)}].$$

We should have at the very least that $\mathbb{P}_\theta(\tau(x) < \infty) = 1$, which happens when $\mathbb{E}_\theta[X] > 0$, equivalently $\kappa'(\theta) > 0$. Then, we have that

$$z(x) = \mathbb{E}_\theta[e^{-\theta S_{\tau(x)} + \tau(x)\kappa(\theta)}].$$

Actually, when a positive solution γ to $\hat{F}(\gamma) = 1$ exists (equivalently $\kappa(\gamma) = 0$), $\theta = \gamma$ gives the efficient Siegmund change of measure. Then, we have that

$$z(x) = \mathbb{E}_\theta[e^{-\theta S_{\tau(x)} + \tau(x)\kappa(\theta)}] = \mathbb{E}_\gamma[e^{-\gamma S_{\tau(x)}}] = e^{-\gamma x} \mathbb{E}_\gamma[e^{-\gamma \xi(x)}],$$

where $\xi(x) = S_{\tau(x)} - x$ is the overshoot.

THEOREM 9.2.

Assume $\mathbb{E}X_1 < 0$, $\mathbb{P}(X_1 > 0) > 0$, $\hat{F}(\gamma) = 1$ for some $\gamma > 0$, and $\hat{F}'(\gamma) < \infty$. For a nonlattice increment distribution, Siegmund's algorithm given by

$$Z(x) = e^{-\gamma x} e^{-\gamma \xi(x)}$$

(simulated in a setting where the distribution of the X 's are exponentially tilted at rate γ) has bounded relative error.

Proof. Under the tilted measure, the overshoot process regenerates at ascending ladder heights and has a limiting overshoot ξ_∞ . Thus $c = \mathbb{E}_\gamma[e^{-\gamma \xi_\infty}] > 0$ and $c_2 = \mathbb{E}_\gamma[e^{-2\gamma \xi_\infty}]$ give

$$z(x) \sim c e^{-\gamma x}, \quad \mathbb{E}_\gamma[Z(x)^2] \sim c_2 e^{-2\gamma x}.$$

Consequently $\text{Var}_\gamma(Z(x))/z(x)^2 \rightarrow c_2/c^2 - 1 < \infty$. □

EXAMPLE 9.3.**PROPOSITION 9.4.**

Among fixed i.i.d. increment changes of measure with positive mean and likelihood ratio $f/\tilde{f}$, the tilted density g_γ is the unique one yielding logarithmic efficiency in Siegmund's algorithm; it also has bounded relative error.

Proof. Sufficiency: the preceding theorem gives bounded relative error for g_γ , which implies logarithmic efficiency. □

References

- [1] Søren Asmussen and Peter W. Glynn. *Stochastic Simulation: Algorithms and Analysis*, volume 57 of *Stochastic Modelling and Applied Probability*. Springer, New York, 2007. Electronic ISBN: 978-0-387-69033-9.